Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. doi: 10.18637/jss.v000.i00

CausalMixGPD: Bayesian Nonparametric Model with Optional Heavy-Tail Augmentaion and Causal Extensions

Arnab Aich
Florida State University

Abstract

Heavy-tailed outcomes are commonly encountered in various scientific settings where rare extreme events dominate risk, cost, and policy recommendations. Although parametric models tend to adequately capture the center of a distribution, they often fail to characterize extreme outcomes, and non-parametric models lack principled extrapolation in tail regions. We develop `CausalMixGPD`, an R package for Bayesian modeling of heavy-tailed outcomes by integrating Dirichlet process mixtures for the center and generalized Pareto distributions for the tails in a unified framework. `CausalMixGPD` allows for both unconditional and covariate-dependent models, enabling flexible models of mixture weights and components as a function of predictors. Posterior computations were carried out using Markov chain Monte Carlo (MCMC) algorithms using the Nimble package in R. The framework is extended by allowing for separate modeling of outcome distributions for treatment groups. `CausalMixGPD` is designed for estimating and predicting causal quantities, such as average and quantile treatment effects, especially when quantiles are extreme. Moreover, this package provides a clustering framework for different kernel specifications. In R data analysis, it has demonstrated commendable performance in estimating extreme causal effects and clustering observations, thus providing a practical and extensible software framework for heavy-tailed outcome modeling.

Keywords: Bayesian nonparametrics; Dirichlet Process Mixture; Generalized Pareto Distribution; Extreme Value Theory; Cluster Analysis; Causal Inference; R; `nimble`.

1. Introduction

Causal inference is often summarized through average effects, but many applications require distributional comparisons between treatment regimes. In clinical trials, investigators may want to understand how a treatment changes outcomes across the outcome scale, including

effects among individuals in the upper tail where events are rare but consequences can be substantial. This shift from mean-based to distribution-based causal questions makes causal analysis tightly linked to how well we can estimate (and predict) outcome distributions under competing treatment assignments.

A growing R ecosystem supports Bayesian causal inference through flexible outcome modeling and posterior uncertainty quantification. For example, bcf implements Bayesian Causal Forests for binary treatments and continuous outcomes, providing posterior inference for both average and heterogeneous treatment effects (bcf 2024). The package bartCause builds causal estimators around Bayesian additive regression trees, using BART as a regression engine for treatment-response modeling and causal effect estimation (bar 2025; Hill 2011). Complementing tree-based approaches, BCEE implements a Bayesian model-averaging strategy for causal effect estimation with support for binary or continuous exposures and outcomes (BCE 2023; Talbot, Lefebvre, Atherton, and Chiu 2015). A recurring practical difficulty in quantile-based causal reporting is the quantile crossing issue, where quantile curves at different probability levels can cross when they are estimated separately. To avoid this, CausalMixGPD directly models outcome densities and derives quantiles and other functionals from the posterior predictive distributions.

In the context of density estimation and conditional density estimation, Dirichlet Process Mixtures (DPM) provide a versatile nonparametric solution because they can capture multimodality and latent heterogeneity without fixing the number of mixing components in advance. In the experimental R ecosystem, Bayesian nonparametric mixture modeling is well supported by software based on Dirichlet process (DP) ideas. For example, dirichletprocess provides a flexible interface for specifying DPMs and fitting them by posterior simulation, yielding posterior inference for mixture weights, component labels, and predictive densities under user-chosen kernels (dir 2023). The package DPpackage offers a broad suite of Bayesian semi- and nonparametric models in R, including DP-based formulations for marginal and conditional densities, regression functions, and model-based predictive distributions (Jara, Hanson, Quintana, Müller, and Rosner 2011). More recently, BNPmix emphasizes practical Bayesian nonparametric mixture approaches, providing posterior inference for clustering structure and density/regression estimation under DP and Pitman–Yor-type mixture specifications (Corradin, Canale, and Nipoti 2021).

That clustering perspective is also important for the present package. Because DP mixtures induce random partitions through latent allocation variables, they naturally support model-based subgroup discovery in addition to density estimation. Bayesian cluster analysis is typically reported through label-invariant summaries such as pairwise co-clustering probabilities (Binder 1978) and representative partitions selected by Dahl's least-squares criterion (Dahl 2006). In regression settings, clustering itself can depend on predictors: dependent Dirichlet process constructions allow covariates to affect mixture weights, component-specific parameters, or both (MacEachern 1999, 2000; De Iorio, Müller, Rosner, and MacEachern 2004; Quintana, Müller, Jara, and MacEachern 2022), and predictor-dependent stick-breaking schemes provide a practical route for covariate-modulated cluster prevalence (Ren, Du, Carin, and Dunson 2011). Existing software supports parts of this workflow, but it typically focuses on bulk density estimation and clustering under a single kernel family without an explicit adjustment for heavy tailed data. As a result, applications that require both latent subgroup discovery and tail extrapolation still need additional modeling outside the standard Bayesian nonparametric mixture workflow.

However, it is important to note that estimating values far in the tail requires special care when data becomes scarce. Additionally, using a single model in the presence of heavy tail data can distort the fit in the central region. In such cases extreme value theory (EVT) provides a principled framework for modeling and extrapolation of tail behavior (de Haan and Ferreira 2007). EVT characterizes the asymptotic distribution of threshold exceedances, which can be used to estimate tail probabilities and quantiles beyond the range of observed data (Balkema and de Haan 1974a). Bayesian implementations of EVT are available in R when posterior uncertainty quantification for tail quantities are required. For example, evdbayes provides Bayesian inference for standard extreme-value models via posterior simulation, enabling posterior and posterior-predictive summaries for extreme-event quantities (evd 2025). For peaks-over-threshold (POT) approaches, the package POT implements a focused collection of tools for GPD modeling and threshold-based tail analysis (POT 2024). Finally, for extreme quantile estimation and rare-event tail summaries, extremefit provides a published software implementation targeted specifically at extreme-quantile estimation (Durrieu, Grama, Jaunatre, Pham, and Tricot 2018).

This article introduces an R package (CausalMixGPD) that connects Bayesian nonparametric modeling with EVT, clustering, and robust causal analyses in the presence of covariates (confounders). The package provides three simultaneous capabilities:

- **Bayesian nonparametric (BNP) density modeling and prediction with optional GPD tail:** CausalMixGPD fits DPMs using a user-chosen kernel below a certain threshold (bulk part), and can optionally splice a generalized Pareto tail above the threshold (tail part) to enable the estimation of tail quantities and posterior predictive inference for rare-event summaries.

- **Clustering under predictor dependent mixtures:** A dedicated clustering interface for both bulk-only and spliced models, supporting weight dependence, parameter dependence, or both, and returning label-invariant clustering summaries through posterior similarity matrices and Dahl representative partitions (Binder 1978; Dahl 2006).

- **Causal estimation and prediction for randomized controlled trials (RCTs) and observational studies:** The final goal of this package is to provide causal estimands by modeling treatment-specific potential outcome densities and reporting contrasts as functionals of the posterior over densities. Framework supports both randomized and observational settings, with the latter using a flexible Bayesian propensity score model to adjust for confounding.

Overall, CausalMixGPD offers a unified workflow for density estimation, prediction, and causal reporting when tail behavior is of significance. The package is implemented in R with computationally intensive components coded in C++ and integrated via the `nimble` framework for model specification and posterior simulation (nim 2025). The remainder of this article is organized as follows: Section 2 provides background on the key modeling components, including DPMs, GPDs,and the spliced bulk-tail construction. Section 3 gives an overview of the package's main functions and user interface. Section 4 and Section 5 illustrates the package's capabilities through applications to causal inference and clustering on R datasets, respectively. Finally, Section 6 concludes with a discussion of limitations and future directions.

2. Background and Preliminaries

Let the outcome–covariate pair be defined by

$$(Y, X) \in \mathbb{R} \times \mathbb{R}^p. \tag{1}$$

For a fixed covariate value x, the conditional cumulative distribution function (CDF) and density are

$$F(y \mid x) = \Pr\,(Y \le y \mid X = x), \qquad f(y \mid x) = \frac{\partial}{\partial y} F(y \mid x), \tag{2}$$

and the conditional quantile function is defined by

$$Q(\tau \mid x) = \inf\Big\{y \in \mathbb{R} : \; F(y \mid x) \ge \tau\Big\}, \qquad 0 < \tau < 1. \tag{3}$$

CausalMixGPD treats conditional distribution of Y given X as the primary inferential object. The conditional density is modeled using a spliced (two part) construction that combines a Non–Parametric model for the center of the distribution where data is substantial and an extreme value distribution for the tail where data is scarce. All posterior summaries reported by the package are functionals of estimated conditional distributions' parameters, including conditional quantiles, densities, or samples.

The following subsections provide background on these two components and their integration into a spliced model.

2.1. Generalized Pareto Distribution for Threshold Exceedances

In modeling terms, the tail region is defined by values that exceed a high threshold. Extreme value theory (EVT) gives a standard approximation for exceedances over such a threshold. Let $u(x)$ be a high threshold, which may depend on x, and define the exceedance

$$Z = Y - u(x), \tag{4}$$

considered conditionally on $Y > u(x)$. The conditional excess distribution is given by

$$F_u(z \mid x) = \Pr\!\big(Y - u(x) \le z \mid Y > u(x), X = x\big), \qquad z \ge 0. \tag{5}$$

Under broad regularity conditions, $F_u(\cdot \mid x)$ is well-approximated by a generalized Pareto distribution (GPD) when $u(x)$ is large (Balkema and de Haan 1974b; Pickands 1975; Coles 2001).

For $y > u(x)$, the corresponding GPD cumulative distribution function is given as

$$F_{\mathrm{GPD}}(y \mid u(x), \sigma(x), \xi) = \begin{cases} 1 - \left(1 + \xi \dfrac{y - u(x)}{\sigma(x)}\right)^{-1/\xi}, & \xi \ne 0, \\ 1 - \exp\!\left(-\dfrac{y - u(x)}{\sigma(x)}\right), & \xi = 0, \end{cases} \tag{6}$$

with support $y \ge u(x)$ when $\xi \ge 0$, and $u(x) \le y \le u(x) - \sigma(x)/\xi$ when $\xi < 0$. The corresponding density function is given by

$$f_{\mathrm{GPD}}(y \mid u(x), \sigma(x), \xi) = \begin{cases} \dfrac{1}{\sigma(x)} \left(1 + \xi \dfrac{y - u(x)}{\sigma(x)}\right)^{-1/\xi - 1}, & \xi \ne 0, \\ \dfrac{1}{\sigma(x)} \exp\!\left(-\dfrac{y - u(x)}{\sigma(x)}\right), & \xi = 0. \end{cases} \tag{7}$$

The associated quantile function is given by

$$Q_{\mathrm{GPD}}(\tau \mid u(x), \sigma(x), \xi) = \begin{cases} u(x) - \sigma(x)\log(1-\tau), & \xi = 0, \\ u(x) + \dfrac{\sigma(x)}{\xi}\Big((1-\tau)^{-\xi} - 1\Big), & \xi \neq 0, \end{cases} \qquad 0 < \tau < 1. \tag{8}$$

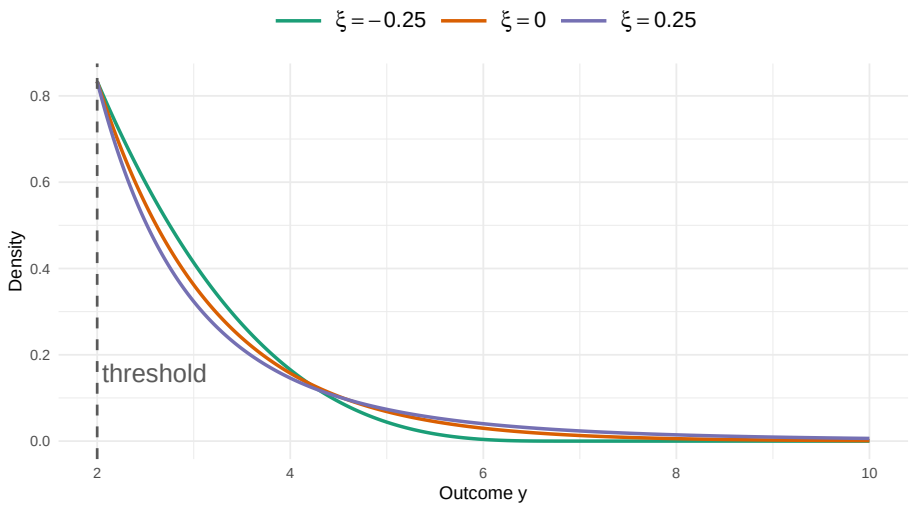

Figure 1: GPD tail density above threshold for multiple shape parameters.

Figure 1, illustrates the GPD density for different shape parameters ξ with a fixed scale parameter σ and threshold u. The shape parameter ξ controls the tail heaviness: $\xi < 0$ corresponds to a light tail with a finite upper bound, $\xi = 0$ corresponds to an exponential tail, and $\xi > 0$ corresponds to a heavy tail with infinite support. The GPD provides a flexible family for modeling exceedances above a high threshold, making it a natural choice for tail modeling in the two part spliced model construction.

Next, we will discuss the modelling of the bulk of the distribution using DPM regression models.

2.2. Dirichlet Process Mixture Model for Conditional Densities

In this package, conditional density of Y given X below a specific threshold ($u(x)$) is modeled using a DPM regression model. Let $k(\cdot \mid x; \theta)$ be a user-chosen kernel family indexed by parameter θ. The hierarchical specification is

$$Y \mid X = x, \theta \sim k(\cdot \mid x; \theta), \qquad \theta \mid H \sim H, \qquad H \mid \kappa, H_0 \sim \mathrm{DP}(\kappa, H_0), \tag{9}$$

where H is an unknown mixing distribution on the kernel-parameter space, H_0 is a base measure, and $\kappa > 0$ is a concentration parameter (Ferguson 1973; Lo 1984). Marginalizing H yields an (almost surely) discrete mixing distribution and an infinite mixture representation:

$$f_{\mathrm{DP}}(y \mid x) = \int k(y \mid x; \theta)\, dH(\theta) = \sum_{j=1}^{\infty} w_j\, k(y \mid x; \theta_j), \qquad w_j \geq 0, \qquad \sum_{j=1}^{\infty} w_j = 1. \tag{10}$$

One representation for the weights is given by Sethuraman's stick-breaking construction (Sethuraman 1994):

$$w_1 = V_1, \qquad w_j = V_j \prod_{\ell<j}(1 - V_\ell), \qquad V_j \overset{\text{iid}}{\sim} \text{Beta}(1, \kappa), \qquad \theta_j \overset{\text{iid}}{\sim} H_0. \tag{11}$$

An equivalent characterization is obtained by marginalizing over H, yielding the Blackwell–MacQueen Pólya urn representation. In particular, the conditional distribution of a parameter value given the others can be written as

$$\theta_i \mid \{\theta_j\}_{j\neq i} \sim \frac{\kappa}{\kappa + n} H_0(\cdot) + \frac{1}{\kappa + n} \sum_{j=1}^{n} \delta_{\theta_j}(\cdot), \tag{12}$$

where $\delta_{\theta_j}(\cdot)$ is a point mass assigned to θ_j . This induces a clustering structure often described via the Chinese restaurant process (CRP) (Blackwell and MacQueen 1973; Neal 2000). Both stick-breaking and CRP representations are widely used for posterior simulation in DPM models (Escobar and West 1995; Neal 2000).

DPM works without specifying the number of mixture components, and can capture complex distributional features such as multimodality, skewness, and heteroscedasticity by choosing an appropriate kernel family. This makes DPMs a natural choice for modeling the bulk of the conditional distribution, where data are abundant and complex features may be present. This package supports multiple kernel family, which is discussed in supplementary materials (12).

2.3. Spliced DPM–GPD Conditional Distribution

CausalMixGPD adopts a spliced (bulk–tail) construction that retains the DPM fit below a high threshold and replaces the upper tail by a GPD component (Frigessi, Haug, and Rue 2002; Behrens, Lopes, and Gamerman 2004; Coles 2001). Let

$$p_u(x) = F_{\text{DP}}(u(x) \mid x; \Theta) \tag{13}$$

denote the bulk probability mass below a threshold ($u(x)$) under the DPM fit. The spliced conditional CDF is given as

$$F(y \mid x; \Theta, \Phi) = \begin{cases} F_{\text{DP}}(y \mid x; \Theta), & y \leq u(x), \\ p_u(x; \Theta) + \{1 - p_u(x; \Theta)\}\, F_{\text{GPD}}(y \mid x; \Phi), & y > u(x), \end{cases} \tag{14}$$

where $F_{\text{DP}}(\cdot)$ is the DPM CDF on the bulk region and $F_{\text{GPD}}(\cdot)$ is the GPD CDF on the exceedance region (Section 2.1). It can be easily verified that the CDF (14) is continuous at $u(x)$ and differentiating (14) yields the spliced density fucntion given as

$$f(y \mid x; \Theta, \Phi) = \begin{cases} f_{\text{DP}}(y \mid x; \Theta), & y \leq u(x), \\ \{1 - p_u(x; \Theta)\}\, f_{\text{GPD}}(y \mid x; \Phi), & y > u(x), \end{cases} \tag{15}$$

where the tail density is a scaled GPD density, with scaling ensuring normalization of $f(\cdot \mid x; \Theta, \Phi)$. The following figure illustrates the spliced density construction, showing the bulk DPM density, the GPD tail density, and the resulting spliced density.

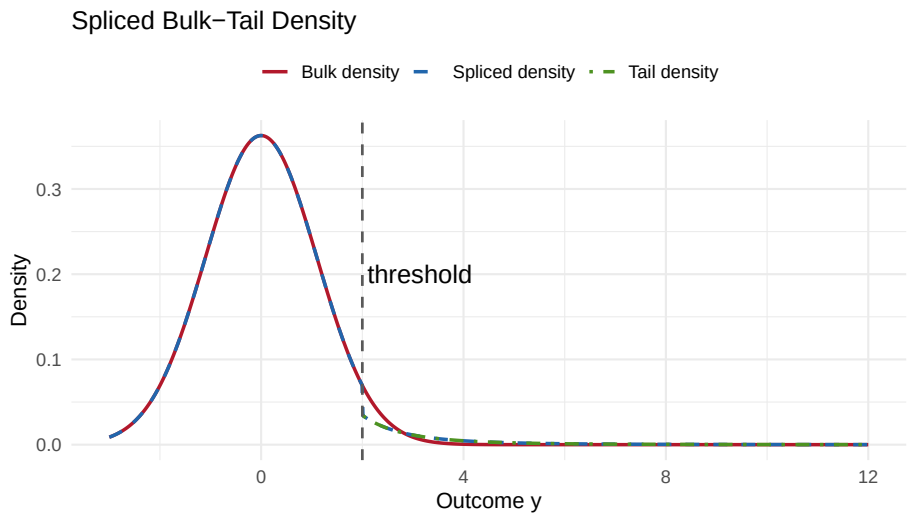

Figure 2: Illustration of the spliced bulk–tail density construction.

To find quantiles below the threshold i.e. for quantile index $\tau \in (0, p_u(x;\Theta)]$, one need to invert the CDF in (14) and obtain quantiles numerically. For $\tau \in (p_u(x;\Theta), 1)$, the index needs to be scaled in terms of threshold excedance probabilty which is defined as

$$\tilde{\tau}(x;\Theta) = \frac{\tau - p_u(x;\Theta)}{1 - p_u(x;\Theta)} \in (0,1), \tag{16}$$

and then the tail quantiles can be obtained by plugging in $\tilde{\tau}(x;\Theta)$ into the GPD quantile function in (8). Thus, the spliced quantile function is given by

$$Q(\tau \mid x;\Theta,\Phi) = \begin{cases} Q_{\mathrm{DP}}(\tau \mid x;\Theta), & \tau \le p_u(x;\Theta), \\ Q_{\mathrm{GPD}}(\tilde{\tau}(x;\Theta) \mid x;\Phi), & \tau > p_u(x;\Theta). \end{cases} \tag{17}$$

To make the spliced form and its associated functionals explicit, the parameters of the bulk and tail components are partitioned into two collections, denoted by Θ and Φ, respectively. Specifically, the bulk distribution defined in Equation (10) can be written as

$$f_{\mathrm{DP}}(y \mid x;\Theta) = \sum_{j=1}^{\infty} w_j \, k(y \mid x;\theta_j(x)) \,, \tag{18}$$

with $\Theta = \{\beta_j\}_{j\ge1}$, where β_j denotes a set of component-specific regression coefficients. The corresponding kernel parameter at covariate value x is given by

$$\theta_j(x) = g(x^\top \beta_j),$$

where $g(\cdot)$ is a user-specified link function. The collection Θ may also include kernel parameters that do not depend on covariates. For example, under a Gaussian kernel, Θ may include regression coefficients β_j^μ along with a component-specific variance parameter σ_j that does not depend on x.

For the tail component, define

$$\Phi = \{\beta_u, \beta_\sigma, \xi\}, \tag{19}$$

where the threshold and scale functions are specified as

$$u(x) = g_u(x^\top \beta_u), \qquad \sigma(x) = g_\sigma(x^\top \beta_\sigma),$$

with $g_u(\cdot)$ and $g_\sigma(\cdot)$ denoting user-specified link functions. The parameter ξ is the GPD shape parameter governing tail heaviness (see Section 2.1).

This package allows users to specify which parameters vary with covariates and which are treated as fixed or entirely determined by prior specifications (see Section 3).

2.4. Clustering under predictor-dependent Bayesian nonparametric mixtures

A common Bayesian nonparametric approach to conditional density modeling represents the collection $\{f(\cdot \mid x) : x \in \mathcal{X}\}$ through predictor-indexed random probability measures. Within the Dependent Dirichlet Process (DDP) framework of MacEachern (1999, 2000), one specifies a family $\{G_x : x \in \mathcal{X}\}$ of random mixing measures whose dependence across x induces flexible borrowing of information in $f(\cdot \mid x)$.

A generic mixture representation is

$$f(y \mid x) \;=\; \int k(y \mid x, \theta)\; dG_x(\theta), \tag{20}$$

where $k(\cdot \mid x, \theta)$ is a kernel and G_x is a predictor-indexed mixing measure. When G_x is discrete (as under stick-breaking constructions), latent parameters exhibit ties, inducing a random partition of observations. Introduce latent cluster-label variables z_i and component parameters $\{\theta_j\}_{j\geq 1}$ such that

$$y_i \mid z_i, \{\theta_j\}, x_i \sim k(\,\cdot \mid x_i, \theta_{z_i})\,. \tag{21}$$

Clustering is then characterized by the posterior distribution of the induced partition $\Pi = \{z_1, \ldots, z_n\}$ and summarized in a label-invariant manner using co-clustering functionals (Binder 1978; Dahl 2006).

In the clustering component considered here, we allow three forms of predictor dependence, each corresponding to a particular construction of G_x (equivalently, predictor dependence in the weights and/or atoms of a stick-breaking representation). In all cases, the conditional density admits the series form

$$f(y \mid x) \;=\; \sum_{j=1}^{\infty} w_j(x)\, k(y \mid x, \theta_j)\,, \tag{22}$$

with $w_j(x) \geq 0$ and $\sum_{j=1}^{\infty} w_j(x) = 1$ for all x.

- **Weight dependence only:** predictors modulate mixture weights while atoms are shared across x. Predictors may affect only the mixing weights, with atoms shared across x:

$$f(y \mid x) \;=\; \sum_{j=1}^{\infty} w_j(x)\, k(y \mid \theta_j)\,. \tag{23}$$

This specification is appropriate when covariates modulate subpopulation prevalence while within-subpopulation distributions remain stable. A common approach uses predictor-dependent stick-breaking, where stick lengths are transformed regressions in x; the logistic stick-breaking process constructs these lengths via logistic regressions and maps them to weights (Ren *et al.* 2011). Such models can be viewed as practical DDP instances in which dependence is introduced through smoothly varying weights (MacEachern 2000; Quintana *et al.* 2022).

- **Atom dependence only:** predictors modulate component-specific kernel parameters while weights are shared across x Alternatively, predictors may enter through component-specific kernel parameters while the weights remain global:

$$f(y \mid x) \;=\; \sum_{j=1}^{\infty} w_j \, k(y \mid x, \theta_j)\,. \tag{24}$$

  Here, θ_j typically indexes cluster-specific regression parameters (with appropriate link functions to enforce constraints), so that clusters correspond to distinct regression surfaces while w_j retains the interpretation of global prevalence. This structure aligns with DDP constructions that share weights while allowing predictor-indexed atoms or atom transformations (MacEachern 1999, 2000), and is closely related to analysis of variance (ANOVA)-type dependent priors for structured collections of random measures (De Iorio *et al.* 2004).

- **Weight and atom dependence:** predictors modulate both mixture weights and component-specific kernel parameters The most flexible specification allows predictors to influence both prevalence and within-cluster behavior:

$$f(y \mid x) \;=\; \sum_{j=1}^{\infty} w_j(x) \, k(y \mid x, \theta_j)\,. \tag{25}$$

  This corresponds to DDP-type models in which both selection probabilities and component-specific conditional structure vary with predictors (Quintana *et al.* 2022).

Posterior clustering summaries are reported using label-invariant functionals of Π. The posterior similarity matrix (PSM) has entries

$$S_{i\ell} \;=\; \Pr(z_i = z_\ell \mid \text{data}), \qquad i, \ell = 1, \ldots, n, \tag{26}$$

estimated by Monte Carlo averages over MCMC output. A representative partition may be obtained by selecting the sampled partition whose adjacency matrix is closest to the PSM under squared loss (Dahl 2006), yielding a stable point estimate while retaining the PSM as the primary uncertainty summary.

2.5. Causal Inference: Notation, Identification, and Reported Estimands

I use the potential outcomes framework (Neyman 1990; Rubin 1974; Imbens and Rubin 2015). Let A denote a binary treatment indicator, let X denote observed pre-treatment covariates, and let Y denote the observed outcome. Each unit has two potential outcomes, $Y(1)$ and $Y(0)$, and the observed outcome satisfies

$$Y = A\,Y(1) + (1 - A)\,Y(0). \tag{27}$$

The treatment-specific conditional distribution functions are the fundamental inferential objects:

$$F_a(y \mid x) = \Pr\{Y(a) \le y \mid X = x\}, \qquad a \in \{0, 1\}. \tag{28}$$

In this package, the corresponding conditional densities are

$$f_a(y \mid x) = \frac{\partial}{\partial y} F_a(y \mid x) \tag{29}$$

where $f_a(y \mid x)$ assumes the spliced DPM structure in (15). Conditional quantiles are defined by inversion of the conditional CDF:

$$Q_a(\tau \mid x) = \inf\{y : F_a(y \mid x) \ge \tau\}, \qquad \tau \in (0, 1). \tag{30}$$

Conditional means are functionals of the conditional density:

$$\mu_a(x) = \mathrm{E}\{Y(a) \mid X = x\} = \int y\, f_a(y \mid x)\, dy. \tag{31}$$

Marginal (standardized) distributions are obtained by averaging conditional distributions over a covariate distribution. The package reports population-standardized and treated-standardized functionals:

$$F_a^{\mathrm{m}}(y) = \mathrm{E}\{F_a(y \mid X)\}, \qquad F_a^{\mathrm{t}}(y) = \mathrm{E}\{F_a(y \mid X) \mid A = 1\}. \tag{32}$$

The corresponding marginal quantiles are

$$Q_a^{\mathrm{m}}(\tau) = \inf\{y : F_a^{\mathrm{m}}(y) \ge \tau\}, \qquad Q_a^{\mathrm{t}}(\tau) = \inf\{y : F_a^{\mathrm{t}}(y) \ge \tau\}. \tag{33}$$

Marginal means are

$$\mu_a^{\mathrm{m}} = \mathrm{E}\{\mu_a(X)\}, \qquad \mu_a^{\mathrm{t}} = \mathrm{E}\{\mu_a(X) \mid A = 1\}. \tag{34}$$

Identification of these quantities depends on the study design. In randomized experiments, randomization together with positivity implies exchangeability between treatment assignment and potential outcomes (Imbens and Rubin 2015). In observational studies, we rely on the following conditions (Rosenbaum and Rubin 1983; Imbens and Rubin 2015):

- **Stable Unit Treatment Value Assumption (SUTVA):** no interference between units and no hidden versions of treatment.
- **Consistency:** if $A = a$, then the observed outcome equals the corresponding potential outcome, $Y = Y(a)$.
- **Conditional ignorability:** treatment is as-if randomized given covariates,

$$\big(Y(0), Y(1)\big) \;\perp\; A \;\mid\; X. \tag{35}$$

- **Overlap (positivity):** each covariate profile has positive probability of receiving either treatment,

$$0 < \Pr\,(A = 1 \mid X = x) < 1 \tag{36}$$

  for covariate values of interest.

Under these conditions, the treatment-specific conditional distributions are identified from the observed data:

$$F_a(y \mid x) = \Pr\left(Y \leq y \mid A = a, X = x\right), \qquad a \in \{0, 1\}. \tag{37}$$

In observational settings, we use the propensity score (PS) to support the design stage (Rosenbaum and Rubin 1983). The PS is the treatment assignment probability given covariates:

$$\rho(x) = \Pr\left(A = 1 \mid X = x\right). \tag{38}$$

A key property of the true propensity score is that conditioning on it balances covariates across treatment groups:

$$A \perp X \mid \rho(X). \tag{39}$$

With estimated scores this balancing property is approximate in finite samples. Following the two-stage strategy described in the thesis, we first estimate $\rho(x)$ using only (A, X) and then augment the outcome model covariates with the estimated score (Rosenbaum and Rubin 1983):

$$\widehat{\rho}(x) \approx \rho(x), \qquad r(x) = \left(x^{\mathsf{T}}, \widehat{\rho}(x)\right)^{\mathsf{T}}. \tag{40}$$

The conditional outcome distributions are then modeled separately within each treatment arm as functions of the augmented predictor $r(x)$, and causal contrasts are computed as functionals of the estimated treatment-specific distributions.

The package reports the following causal estimands:

- **Average Treatment Effect (ATE):**

$$\mathrm{ATE} = \mu_1^{\mathrm{m}} - \mu_0^{\mathrm{m}}. \tag{41}$$

- **Average Treatment Effect on the Treated (ATT):**

$$\mathrm{ATT} = \mu_1^{\mathrm{t}} - \mu_0^{\mathrm{t}}. \tag{42}$$

- **Quantile Treatment Effect (QTE):** for any $\tau \in (0, 1)$ (Firpo 2007),

$$\mathrm{QTE}(\tau) = Q_1^{\mathrm{m}}(\tau) - Q_0^{\mathrm{m}}(\tau). \tag{43}$$

- **Quantile Treatment Effect on the Treated (QTT):** for any $\tau \in (0, 1)$,

$$\mathrm{QTT}(\tau) = Q_1^{\mathrm{t}}(\tau) - Q_0^{\mathrm{t}}(\tau). \tag{44}$$

- **Conditional Average Treatment Effect (CATE):**

$$\mathrm{CATE}(x) = \mu_1(x) - \mu_0(x). \tag{45}$$

- **Conditional Quantile Treatment Effect (CQTE):** for any $\tau \in (0, 1)$,

$$\mathrm{CQTE}(\tau \mid x) = Q_1(\tau \mid x) - Q_0(\tau \mid x). \tag{46}$$

This section established the main probabilistic and causal building blocks used in the remainder of the work. I first defined conditional distributional targets and introduced a DPM model to flexibly estimate bulk conditional outcome densities. I then reviewed generalized Pareto modeling for threshold exceedances and constructed a spliced DPM–GPD conditional distribution that preserves the estimated bulk while enabling principled tail extrapolation. Finally, we introduced the potential outcomes framework and the treatment-effect estimands reported in the package, showing how marginal and conditional effects can be obtained as functionals of treatment-specific conditional distributions (with propensity-score augmentation supporting confounding adjustment in observational studies). Subsequent chapters build on these preliminaries to specify the full hierarchical model, posterior computation, and empirical evaluation of tail-aware distributional causal effects.

3. Package Overview: Workflow, Model, and Inference

This section describes the main workflow provided by CausalMixGPD for one-arm conditional modeling and prediction when a generalized Pareto tail is spliced above a covariate-dependent threshold. The causal extension reuses the same modeling, estimation, and prediction machinery after introducing treatment-specific outcome models and a propensity score component; we defer those additions to the causal section.

3.1. Augmented DPM–GPD model, likelihood, and priors

Let the observed data be

$$\mathcal{D} = \{(y_i, x_i)\}_{i=1}^n,$$

```
data("nc_posX100_p3_k2", package = "CausalMixGPD")
dat <- data.frame(y = nc_posX100_p3_k2$y, nc_posX100_p3_k2$X)
```

with $x_i \in \mathbb{R}^p$ denoting the covariate vector (including an intercept if desired). The target conditional distribution $F(\cdot \mid x)$ and its spliced DPM–GPD construction are defined in Section 2, with the spliced CDF and density given in (14)–(15). Posterior computation in CausalMixGPD approximates the bulk DPM by a finite truncation with J mixture components,

$$f_{\mathrm{DP}}(y \mid x; \Theta) \;\approx\; \sum_{j=1}^{J} w_j \, k(y \mid x; \theta_j), \qquad j = 1, \ldots, J,$$

where j indexes mixture components, $w_j \geq 0$, and $\sum_{j=1}^{J} w_j = 1$. Kernel parameters can be constant or covariate-dependent through link specifications of the form

$$g(\psi_j(x)) = x^\top \beta_{j,\psi}, \tag{47}$$

where $g(\cdot)$ enforces parameter support (e.g., positive scale parameters).

```
mcmc_fixed
```

```
fit <- dpmgpd(
  formula = y ~ x1 + x2 + x3,
  data = dat,
  backend = "sb",
  kernel = "lognormal",
  components = 5,
  mcmc = mcmc_fixed
)
```

For the spliced model, let $u(x)$ be the threshold, $\sigma(x) > 0$ the tail scale, and $\xi \in \mathbb{R}$ the tail shape (as in Section 2.3). Define $\delta_i = \mathbf{1}\{y_i > u(x_i)\}$. Using the spliced density (15) with the truncated bulk mixture, the likelihood is

$$L(\Theta, \Phi; \mathcal{D}) = \prod_{i=1}^{n} f(y_i \mid x_i; \Theta, \Phi), \tag{48}$$

where Θ collects the truncation-level bulk parameters $\{w_j, \theta_j\}_{j=1}^{J}$, and Φ collects the tail parameters governing $u(x)$, $\sigma(x)$, and ξ. Equivalently, the log-likelihood can be written as

$$\begin{aligned} \log L(\Theta, \Phi; \mathcal{D}) \;=\; & \sum_{i=1}^{n} (1 - \delta_i) \log f_{\mathrm{DP}}(y_i \mid x_i; \Theta) \\ & + \sum_{i=1}^{n} \delta_i \Big[\log\{1 - F_{\mathrm{DP}}(u(x_i) \mid x_i; \Theta)\} + \log f_{\mathrm{GPD}}(y_i \mid x_i; \Phi) \Big], \end{aligned} \tag{49}$$

where f_{GPD} is given in (7) and $F_{\mathrm{DP}}(\cdot \mid x)$ is the bulk mixture CDF induced by the truncated mixture.

Priors follow the decomposition implied by (11) and (12). Under the SB backend, the truncation-level weights are generated through Beta stick breaks (Section 2, (11)), which are represented in nimble using the Beta distribution in model code (see `dbeta`). Under the CRP backend, the latent-cluster-label representation is implemented using the CRP distribution in nimble (see `dCRP`), matching the predictive representation in (12). Kernel parameters and tail parameters are assigned weakly informative priors, with regression coefficients $\beta_{j,\psi}$ arising through the link representation (47). The same link construction is used for covariate-dependent $u(x)$ and $\sigma(x)$ when those are modeled as functions of x.

3.2. MCMC algorithm, convergence diagnostics, and parameter extraction

Posterior simulation targets $p(\Theta, \Phi \mid \mathcal{D}) \propto L(\Theta, \Phi; \mathcal{D})\, p(\Theta, \Phi)$ using the likelihood (48) and the prior structure implied by the SB or CRP representation (Section 2, (11)–(12)). Given a model specification, CausalMixGPD runs MCMC through nimble (de Valpine, Turek, Paciorek, Anderson-Bergman, Temple Lang, and Bodik 2017; nim 2025) under the chosen backend; SB uses Beta stick breaks in model code (see `dbeta`) and CRP uses the CRP distribution for latent cluster labels (see `dCRP`). Users control the number of iterations, burn-in, thinning, and number of chains through the `mcmc` argument in the wrappers.

Convergence checking is supported through standard multi-chain diagnostics and trace-based visualizations. The model object supports `summary()` for posterior summaries, `plot()` for

diagnostic plots (trace, running mean, autocorrelation, and multi-chain diagnostics such as $\widehat{R}$ when multiple chains are used), and `params()` as a lightweight extractor returning posterior mean parameters reshaped to natural dimensions. Interval summaries used by `predict()` can be chosen as equal-tailed credible intervals or HPD intervals as described in Section 3.3 (Gelman, Carlin, Stern, Dunson, Vehtari, and Rubin 2013; Plummer, Best, Cowles, and Vines 2024). Diagnostic plotting relies on established MCMC diagnostic tooling (ggm 2025).

```
print(fit)
p_hat <- params(fit)
p_hat
```

Bulk-only special case (no GPD tail)

Setting `GPD=FALSE` removes the splicing step so that $f(y \mid x; \Theta) \equiv f_{\mathrm{DP}}(y \mid x; \Theta)$, and $L(\Theta, \Phi; \mathcal{D})$ reduces to the bulk-mixture likelihood in Θ. All estimation and prediction targets described below remain available, with tail-specific terms omitted. The wrapper `dpmix()` fits this bulk-only model, while `dpmgpd()` fits the augmented (spliced) model.

3.3. Posterior functionals, interval summaries and prediction

The estimands reported by CausalMixGPD are functionals of the estimated conditional distribution $F(\cdot \mid x)$ from Section 2. In this article we focus on four conditional functionals: density $f(y \mid x)$, survival $S(y \mid x) = 1 - F(y \mid x)$, quantiles $Q(\tau \mid x)$, and mean-type summaries.

Let $(\Theta^{(s)}, \Phi^{(s)})_{s=1}^{S}$ denote post-burn-in draws from the posterior. For density, survival, and quantiles, CausalMixGPD evaluates the corresponding draw-wise functional using the spliced formulas (14)–(17) (with the truncated bulk mixture), and then summarizes across draws. Specifically, for a generic scalar functional $T(\Theta, \Phi; x)$ (e.g., $f(y_0 \mid x)$ or $Q(\tau \mid x)$),

$$\widehat{T}(x) = \frac{1}{S} \sum_{s=1}^{S} T(\Theta^{(s)}, \Phi^{(s)}; x),$$

and uncertainty is summarized either by equal-tailed credible intervals (posterior quantiles) or by highest posterior density (HPD) intervals (shortest intervals covering the target posterior mass). Equal-tailed intervals and general Bayesian posterior summaries follow standard practice (Gelman *et al.* 2013); HPD intervals are computed from the MCMC draws using established implementations (Plummer *et al.* 2024).

Mean-type summaries are handled differently in the package. For `type="mean"` and `type="rmean"`, CausalMixGPD uses posterior predictive Monte Carlo within each posterior draw rather than direct plug-in evaluation. For a fixed covariate value x, within draw s the package generates

$$y_1^{(s)}, \ldots, y_M^{(s)} \;\sim\; f(\cdot \mid x; \Theta^{(s)}, \Phi^{(s)}),$$

and computes

$$\mu^{(s)}(x) = \frac{1}{M} \sum_{m=1}^{M} y_m^{(s)}, \qquad \mu_c^{(s)}(x) = \frac{1}{M} \sum_{m=1}^{M} \min\{y_m^{(s)}, c\},$$

corresponding to the conditional mean and the restricted mean at cutoff c, respectively. Posterior summaries and intervals are then formed from $\{\mu^{(s)}(x)\}_{s=1}^{S}$ or $\{\mu_c^{(s)}(x)\}_{s=1}^{S}$. Under the spliced model, the conditional mean is infinite when $\xi \geq 1$; accordingly, CausalMixGPD flags `type="mean"` as infinite when there is posterior mass with $\xi \geq 1$, while `type="rmean"` remains finite by construction.

Using the same fitted model `fit` from Section 3.1, we now compute posterior summaries on the original training covariates (default behavior of `predict()` when `newdata` is not supplied).

```
q_grid <- c(0.50, 0.90, 0.95, 0.99)
x_eval <- dat[1:20, c("x1", "x2", "x3")]

est_quant <- predict(fit, newdata = x_eval, type = "quantile", index = q_grid,
                     interval = "hpd", level = 0.95)
```

For a new covariate value $x^\star$, prediction is based on the posterior predictive distribution

$$p(y^\star \mid x^\star, \mathcal{D}) = \int f(y^\star \mid x^\star; \Theta, \Phi)\, p(\Theta, \Phi \mid \mathcal{D})\, d\Theta\, d\Phi.$$

The `predict()` method returns posterior summaries for the same functional targets as in Section 3.3. For `type="density"` and `type="survival"`, the user supplies a grid of y values and the method returns pointwise posterior means with interval bounds computed draw-by-draw. For `type="quantile"`, the user supplies quantile indices τ and the method returns posterior summaries of $Q(\tau \mid x^\star)$ computed using the spliced quantile representation in (17). For `type="mean"` and `type="rmean"`, prediction uses posterior predictive Monte Carlo as described above, with x replaced by $x^\star$ and with user-controlled M through `nsim_mean`.

```
q_levels <- c(0.25, 0.50, 0.75)
x_new <- as.data.frame(lapply(
  dat[, c("x1", "x2", "x3")],
  quantile,
  probs = q_levels,
  na.rm = TRUE
))
rownames(x_new) <- c("q25", "q50", "q75")
y_grid <- seq(0, 10, length.out = 200)
p_grid <- c(0.50, 0.90, 0.95, 0.99)
```

```
pdens <- predict(fit, newdata = x_new, y = y_grid, type = "density",
                 interval = "credible", level = 0.95)
```

```
psurv <- predict(fit, newdata = x_new, y = y_grid, type = "survival",
                 interval = "credible", level = 0.95)
```

```
pquant <- predict(fit, newdata = x_new, type = "quantile", index = p_grid,
                  interval = "hpd", level = 0.95)
```

3.4. Clustering extension

The mixture modeling framework introduced earlier induces a latent partition of the observations through the latent cluster labels z_i. While the primary modeling workflow of the package focuses on estimating conditional densities and causal functionals, the clustering extension instead treats this latent partition as the main object of interest. The package therefore provides a dedicated clustering interface that fits the same mixture models but returns clustering summaries derived from the posterior distribution of the latent cluster labels.

The clustering workflow mirrors the design used throughout the package. Users specify the response variable, optional covariates, the mixture kernel, and whether the model should use the bulk-only or the spliced likelihood. The resulting model is then fitted using the same MCMC framework used for the conditional and causal models. Consequently, clustering models share the same prior specification, kernel choices, and computational infrastructure as the rest of the package.

Two clustering wrappers are provided. The first constructs mixture models using only the bulk kernel distribution. The second uses the spliced bulk–tail likelihood introduced earlier, allowing the clustering structure to account for extreme observations through the generalized Pareto tail. Both interfaces accept the same model specification arguments as the main modeling functions, ensuring that clustering models can be constructed using the same syntax and options.

Three forms of covariate dependence are supported in clustering models. First, covariates may influence the mixture weights while the component distributions remain invariant across observations. In this configuration, clusters represent subpopulations whose prevalence varies with predictors. Second, the mixture weights may remain global while the component parameters depend on covariates through the same link structures used in the conditional density models. In this case clusters represent groups with distinct covariate-response relationships. Third, both the mixture weights and the component parameters may depend on covariates simultaneously, allowing clusters to capture both changing prevalence and covariate-dependent distributional behavior.

Posterior inference for clustering models uses the same sampling procedure as the main mixture models, jointly updating the component parameters and the latent cluster labels. Let $z^{(s)} = (z_1^{(s)}, \ldots, z_n^{(s)})$, $s = 1, \ldots, S$, denote the retained MCMC draws of the latent cluster-label vector after burn-in and thinning. Because mixture components are only identified up to label permutation, `CausalMixGPD` reports label-invariant clustering summaries based on pairwise co-clustering probabilities and a representative partition (Binder 1978; Dahl 2006).

The primary uncertainty summary is the posterior similarity matrix (PSM), computed entrywise as

$$\widehat{S}_{i\ell} = \frac{1}{S} \sum_{s=1}^{S} \mathbf{1}\Big\{ z_i^{(s)} = z_\ell^{(s)} \Big\}, \qquad i, \ell = 1, \ldots, n.$$

Thus, $\widehat{S}_{i\ell}$ is the posterior probability that observations i and ℓ belong to the same cluster.

A single representative partition is then extracted by selecting the sampled partition whose adjacency matrix is closest to the PSM under squared loss (Dahl 2006):

$$s^\star = \arg\min_{1\le s\le S} \sum_{i=1}^{n}\sum_{\ell=1}^{n} \left(\mathbf{1}\left\{z_i^{(s)} = z_\ell^{(s)}\right\} - \widehat{S}_{i\ell}\right)^2 .$$

The representative labels are $\hat{z}_i = z_i^{(s^\star)}$, relabeled to consecutive integers $1,\ldots,K$, and the reported cluster sizes are $n_k = \sum_{i=1}^{n} \mathbf{1}\{\hat{z}_i = k\}$.

When `return_scores = TRUE`, the package also returns a training membership score matrix. If $C_k = \{\ell : \hat{z}_\ell = k\}$ denotes cluster k in the representative partition, the unnormalized score for observation i and cluster k is

$$q_{ik} = \frac{1}{|C_k|} \sum_{\ell\in C_k} \widehat{S}_{i\ell},$$

and the normalized training score is

$$p_{ik} = \frac{q_{ik}}{\sum_{h=1}^{K} q_{ih}}.$$

The hard representative label is $\arg\max_{1\le k\le K} p_{ik}$, while a simple cluster-certainty summary is $\max_{1\le k\le K} p_{ik}$.

For a new observation $(y_i^{\text{new}}, x_i^{\text{new}})$, the package first forms within-draw component scores

$$r_{ij}^{(s)} \propto \omega_j^{(s)}(x_i^{\text{new}})\, f_j^{(s)}(y_i^{\text{new}} \mid x_i^{\text{new}}), \qquad j = 1,\ldots,J,$$

where $\omega_j^{(s)}(x)$ is the draw-specific gating weight when that quantity is available; otherwise the implementation falls back to sampled global weights, when present, or to the empirical component frequencies in draw s. For occupied components, these scores are mapped to representative clusters through

$$m_{jc}^{(s)} = \frac{1}{n_j^{(s)}} \sum_{\ell:\, z_\ell^{(s)}=j} \mathbf{1}\{\hat{z}_\ell = c\}, \qquad c = 1,\ldots,K,$$

and then averaged over posterior draws:

$$p_{ic}^{\text{new}} = \frac{1}{S}\sum_{s=1}^{S}\sum_{j=1}^{J} r_{ij}^{(s)} m_{jc}^{(s)}.$$

Empty sampled components are assigned a uniform map over the K representative clusters. These are the quantities returned internally by `predict.dpmixgpd_cluster_fit()`. After row normalization, the predicted cluster label is $\arg\max_{1\le c\le K} p_{ic}^{\text{new}}$.

The clustering workflow is illustrated below using one of the example data sets included with the package.

```
data("nc_realX100_p3_k2", package = "CausalMixGPD")

dat_cl <- data.frame(y = nc_realX100_p3_k2$y, nc_realX100_p3_k2$X)

fit_cluster <- dpmix.cluster(
  y ~ x1 + x2 + x3,
  data       = dat_cl,
  kernel     = "normal",
  type       = "param",
  components = 8,
  mcmc = mcmc_fixed
)
```

Posterior clustering summaries can be obtained directly from the fitted object. The following code returns the PSM, the representative training partition, the training cluster sizes, and the associated certainty scores.

```
cluster_psm   <- predict(fit_cluster, type = "psm")
cluster_train <- predict(fit_cluster, type = "label", return_scores = TRUE)

cluster_sizes <- data.frame(
  cluster = names(summary(cluster_train)$cluster_sizes),
  n = as.integer(summary(cluster_train)$cluster_sizes)
)
cluster_certainty <- apply(cluster_train$scores, 1, max)
```

Cluster assignments for new observations are obtained from the posterior predictive score matrix in the same interface.

```
cluster_new <- predict(
  fit_cluster,
  newdata = dat_cl,
  type    = "label",
  return_scores = TRUE
)

cluster_new_certainty <- apply(cluster_new$scores, 1, max)
```

The same workflow applies to the spliced mixture model, which incorporates the generalized Pareto tail component when clustering data with heavy-tailed behavior.

```
fit_cluster_spliced <- dpmgpd.cluster(
  y ~ x1 + x2 + x3,
  data       = dat_cl,
  kernel     = "normal",
  type       = "both",
```

```
  components = 8,
  mcmc = mcmc_fixed
)
```

3.5. Causal extension and propensity score augmentation

This package extends the one-arm conditional mixture regression framework to two-arm causal studies by fitting an outcome model under treatment and under control, optionally augmenting the outcome covariates with a PS summary. Throughout this section we write the model in its default *augmented spliced* form (bulk mixture + GPD tail), using the same conditional distributional building blocks introduced earlier (so we do not restate the spliced density; see (15) and the corresponding conditional functionals defined in the previous sections).

Data, model components, and likelihood factorization

Let

$$\mathcal{D} = \{(y_i, a_i, \boldsymbol{x}_i)\}_{i=1}^n, \qquad a_i \in \{0, 1\},$$

where $a_i = 1$ denotes treatment and $a_i = 0$ denotes control. The causal interface consists of two pieces.

First, when PS augmentation is enabled, a binary regression model is used for the treatment mechanism

$$a_i \mid \boldsymbol{x}_i, \boldsymbol{\gamma} \sim \text{Bernoulli}(e_{\boldsymbol{\gamma}}(\boldsymbol{x}_i)), \tag{50}$$

$$h(e_{\boldsymbol{\gamma}}(\boldsymbol{x}_i)) = \gamma_0 + \boldsymbol{x}_i^\top \boldsymbol{\gamma}, \qquad h \in \{\text{logit}, \text{probit}\}, \tag{51}$$

which is consistent with the definition and balancing property in (38)–(39). The package also supports a `"naive"` option, in which $e(\boldsymbol{x})$ is taken to be constant (no covariate dependence).

Second, the outcome model is fit separately within each arm. Denote by $F_a(\cdot \mid \boldsymbol{r})$ the fitted conditional outcome distribution under arm $a \in \{0, 1\}$, where $\boldsymbol{r}$ is the covariate vector used by the outcome model. With PS augmentation, the package constructs an *augmented* covariate vector

$$\boldsymbol{r}(\boldsymbol{x}) = \Big(1,\ \boldsymbol{x}^\top,\ \psi(\widehat{e}(\boldsymbol{x}))\Big)^\top, \tag{52}$$

where $\widehat{e}(\boldsymbol{x})$ is a posterior summary of the PS (posterior mean or median, controlled by `ps_summary`), and $\psi(\cdot)$ is either $\psi(p) = \text{logit}(p)$ or $\psi(p) = p$ (controlled by `ps_scale`). When PS augmentation is disabled, $\boldsymbol{r}(\boldsymbol{x}) = (1, \boldsymbol{x}^\top)^\top$.

Within each arm, the conditional density $f_a(y \mid \boldsymbol{r})$ follows the same mixture regression + optional GPD tail specification described in the one-arm section (kernel choice, link structure $g(\cdot)$ for covariate-dependent parameters, and either `"sb"` or `"crp"` backend for the DPM). Let $\boldsymbol{\Theta}_a$ collect the arm-specific mixture and tail parameters. The arm-wise likelihood contributions are

$$L_a(\boldsymbol{\Theta}_a; \mathcal{D}) = \prod_{i:\, a_i = a} f_a(y_i \mid \boldsymbol{r}(\boldsymbol{x}_i); \boldsymbol{\Theta}_a)\,, \qquad a \in \{0, 1\},$$

with $f_a(\cdot \mid \cdot)$ given by the spliced construction already defined (bulk mixture below threshold and GPD exceedances above threshold). Practically, the implementation uses a two-stage

workflow: (i) fit the PS model using (50)–(51), then (ii) compute $\widehat{e}(\boldsymbol{x}_i)$ and build $\boldsymbol{r}(\boldsymbol{x}_i)$ in (52) before fitting the two arm-specific outcome models.

Prior specification (PS then outcome)

For the PS regression coefficients, the default prior is independent Normal,

$$\gamma_0 \sim \mathcal{N}(m_0, s_0^2), \qquad \gamma_k \sim \mathcal{N}(m_0, s_0^2),$$

with defaults controlled by `ps_prior`.

For each arm's outcome model, priors follow the same structure as the one-arm case: a DPM prior (with either truncated stick-breaking or a CRP latent-cluster-label representation, implemented in `nimble` (de Valpine *et al.* 2017; nim 2025)) and kernel-/tail-specific priors on the bulk and GPD parameters. Since the causal fit is simply a pair of one-arm fits (treated and control), all customization options for kernels, links, and tail inclusion apply arm-wise. For quick lookup of these controls and their code-level defaults, see the supplementary workflow maps (Figures 8 and 9) and option matrices in Tables 11–16.

MCMC algorithm, diagnostics, and parameter extraction

The causal fit runs MCMC in two stages, each with its own iteration controls.

(i) PS stage: posterior simulation for $\boldsymbol{\gamma}$ in (50)–(51) using `mcmc_ps` settings. The PS posterior draws are summarized to obtain $\widehat{e}(\boldsymbol{x}_i)$, which is optionally transformed via `ps_scale` and appended to the outcome covariates as in (52).

(ii) Outcome stage: arm-specific posterior simulation for $\boldsymbol{\Theta}_0$ and $\boldsymbol{\Theta}_1$ using `mcmc_outcome` settings, with either `"sb"` or `"crp"` backend.

For routine convergence checks, users can inspect trace plots and autocorrelation for monitored parameters and (when running multiple chains) compare between-chain and within-chain mixing. The package stores MCMC samples in the fitted object, and standard diagnostics can be applied after converting samples to `coda` objects (e.g., effective sample size, Geweke diagnostics, and HPD intervals via `coda::HPDinterval`; see the `coda` reference at the end of this message). Posterior parameter summaries can be extracted via the exported `params()` method, which returns arm-specific parameter tables for causal fits.

Estimation and interval summaries for causal contrasts

All reported causal quantities are computed as functionals of the fitted arm-specific conditional distributions. Let $T_a(\boldsymbol{x})$ denote any conditional functional under arm a (density, survival, quantile, or mean/restricted mean), computed from $F_a(\cdot \mid \boldsymbol{r}(\boldsymbol{x}))$ using the same definitions as in the one-arm case. The corresponding conditional causal contrast is

$$\Delta_T(\boldsymbol{x}) = T_1(\boldsymbol{x}) - T_0(\boldsymbol{x}).$$

The package computes $\Delta_T(\boldsymbol{x})$ *draw-by-draw* from posterior samples, then summarizes $\Delta_T(\boldsymbol{x})$ by a posterior mean/median and uncertainty intervals. Equal-tailed credible intervals are formed by posterior quantiles, while HPD intervals are computed using `coda::HPDinterval`.

The main causal estimators provided by the package are:

(a) Conditional effects at covariate profiles. For a user-supplied set of covariate rows $\{\boldsymbol{x}_m^\star\}_{m=1}^M$,

$$\mathrm{CATE}(\boldsymbol{x}_m^\star) = \mu_1(\boldsymbol{x}_m^\star) - \mu_0(\boldsymbol{x}_m^\star), \qquad \mathrm{CQTE}(\tau \mid \boldsymbol{x}_m^\star) = Q_1(\tau \mid \boldsymbol{x}_m^\star) - Q_0(\tau \mid \boldsymbol{x}_m^\star),$$

where $\mu_a(\boldsymbol{x}) = \mathbb{E}(Y \mid A = a, \boldsymbol{X} = \boldsymbol{x})$ and $Q_a(\tau \mid \boldsymbol{x})$ is the conditional quantile function. In heavy-tailed settings, $\mu_a(\boldsymbol{x})$ can be unstable when the implied tail shape yields infinite mean; the package therefore supports both `type="mean"` and `type="rmean"` for (restricted) mean-based effects, with restricted means computed as $\mathbb{E}[\min\{Y, c\} \mid A = a, \boldsymbol{X} = \boldsymbol{x}]$ at user-specified cutoff c.

(b) Marginal (standardized) summaries. The package reports marginal ATE and marginal QTE-style summaries by empirical standardization over covariate profiles:

$$\mathrm{ATE} = \frac{1}{n}\sum_{i=1}^n \Big\{\mu_1(\boldsymbol{x}_i) - \mu_0(\boldsymbol{x}_i)\Big\}, \tag{53}$$

$$\mathrm{QTE}(\tau) = Q_1^{\mathrm{m}}(\tau) - Q_0^{\mathrm{m}}(\tau), \qquad Q_a^{\mathrm{m}}(\tau) = \inf\{y : F_a^{\mathrm{m}}(y) \geq \tau\}, \quad F_a^{\mathrm{m}}(y) = \frac{1}{n}\sum_{i=1}^n F_a(y \mid \boldsymbol{x}_i). \tag{54}$$

Treated-standardized analogues (ATT, QTT) are obtained by replacing the full-sample covariate empirical distribution with the treated-subsample empirical distribution $\{i : a_i = 1\}$. In the current implementation, `ate()` and `att()` aggregate draw-wise conditional mean contrasts across rows, while `qte()` and `qtt()` compute draw-wise arm-specific marginal quantiles and then contrast them.

Posterior prediction at new covariate profiles

Prediction in the causal setup proceeds by producing arm-specific predictive summaries at new covariates and then differencing when required. For new covariates $\boldsymbol{x}^\star$, the package first predicts the PS (if enabled), forms $\boldsymbol{r}(\boldsymbol{x}^\star)$ via (52), and then evaluates the desired distributional functional under both arms. For `type = "mean"` and `type = "quantile"`, `predict()` returns treated-minus-control summaries and stores arm-specific predictions as attributes; for `type = "density"`, `"survival"`, and `"prob"`, it returns arm-specific summaries directly.

A recommended way to build an interpretable prediction grid is to use coordinate-wise empirical quartiles. Let $q_{k,p}$ be the pth sample quantile of covariate x_k for $p \in \{0.25, 0.5, 0.75\}$. The grid

$$\mathcal{X}_\star = \prod_{k=1}^p \{q_{k,0.25}, q_{k,0.5}, q_{k,0.75}\}$$

yields 3^p representative covariate profiles at which CATE, CQTE, survival contrasts, and other predictive contrasts can be reported.

User-facing workflow (code)

The following example uses the formula interface and the high-level wrapper, fitting the augmented spliced model by default.

```
data("causal_pos500_p3_k2", package = "CausalMixGPD")
dat <- causal_pos500_p3_k2
df  <- data.frame(y = dat$y, A = dat$A, dat$X)
```

```
cfit <- dpmgpd.causal(
  formula = y ~ x1 + x2 + x3,
  data = df,
  treat = "A",
  backend = "sb",
  kernel = "lognormal",
  components = 5,
  PS = "logit",
  ps_scale = "logit",
  ps_summary = "mean",
  mcmc_outcome = mcmc_fixed,
  mcmc_ps = mcmc_fixed
)
```

The fitted model object `cfit` contains the MCMC samples for both the PS and the arm-specific outcome models, and can be used to compute causal estimands and predictive summaries.

To estimate the ATE and QTE at the empirical covariate distribution, use the `ate()` and `qte()` functions, which compute the standardized contrasts in (53) and (54).

```
ate_hat <- ate(cfit, level = 0.90, interval = "credible")
qte_hat <- qte(cfit, probs = c(0.50, 0.90, 0.95), level = 0.90, interval = "hpd")
```

For subpopulation effects, the `cate()` and `cqte()` functions compute conditional contrasts at user-specified covariate rows.

```
qs    <- c(0.25, 0.50, 0.75)
Xgrid <- expand.grid(lapply(df[c("x1","x2","x3")], quantile, probs = qs))
```

The following code then produces conditional quantile and survival predictions at the grid points, returning treated-minus-control contrasts for quantiles and arm-specific summaries for survival.

```
pred_q <- predict(cfit, newdata = Xgrid, type = "quantile", p = c(0.50, 0.90))
pred_s <- predict(cfit, newdata = Xgrid, type = "survival", y = rep(4, nrow(Xgrid)))
```

Bulk-only causal mixtures (no GPD tail)

To run the same two-arm regression without a GPD tail, use the bulk-only wrapper `dpmix.causal()` (or set `GPD=FALSE` at bundle construction). All causal estimand and prediction functions (`ate`, `qte`, `cate`, `cqte`, and `predict`) operate the same way, now interpreting $F_a(\cdot \mid \boldsymbol{x})$ as the bulk mixture distribution.

4. Data analysis I: Clustering

This application demonstrates the clustering wing for a conditional mixture model under a bulk-only likelihood (no GPD tail). The objective is to infer latent groups of observational units implied by the fitted mixture model, and to summarize posterior clustering uncertainty in a label-invariant way. Throughout, clustering is reported via (i) the posterior similarity matrix (PSM) and (ii) a single representative partition derived from the posterior draws, together with out-of-sample cluster label prediction for new units.

4.1. Dataset and objective

We use the `Boston` housing data (MASS), treating each census tract as an observational unit. Let

$$\mathcal{D} = \{(y_i, \boldsymbol{x}_i)\}_{i=1}^n,$$

where y_i is the per-capita crime rate (`crim`) for tract i, and $\boldsymbol{x}_i$ is a vector of tract characteristics. Our aim is twofold: (i) to cluster tracts using a flexible conditional mixture model that can accommodate skewness and heterogeneity, and (ii) to assign cluster labels to new tracts using posterior predictive co-clustering evidence.

4.2. Preprocessing and train/test split

We split the data into a training set used to fit the clustering model and a test set used only for cluster-label prediction. Covariates are standardized using training-set moments to avoid leakage.

```r
library(MASS)

set.seed(123)
data("Boston", package = "MASS")

dat <- Boston
dat <- subset(dat, select = c(crim, lstat, rm, nox))

n <- nrow(dat)
idx_train <- sample(seq_len(n), size = floor(0.75 * n), replace = FALSE)
train_dat <- dat[idx_train, ]
test_dat  <- dat[-idx_train, ]

# standardize covariates using training set
xvars <- c("lstat", "rm", "nox")
mu_x  <- vapply(train_dat[, xvars], mean, numeric(1))
sd_x  <- vapply(train_dat[, xvars], sd,   numeric(1))

train_dat[, xvars] <- sweep(sweep(train_dat[, xvars], 2, mu_x, "-"), 2, sd_x, "/")
test_dat[, xvars]  <- sweep(sweep(test_dat[, xvars],  2, mu_x, "-"), 2, sd_x, "/")
```

```
##   sample   n
## 1  train 379
## 2   test 127
```

4.3. Clustering model specification and fit

We fit a bulk-only conditional mixture model for `crim` with three covariates. In this illustration, we select `type="param"`, meaning covariates enter through component-specific regression parameters while the mixture weights are global. This configuration clusters observations by their regression behavior and distributional differences while keeping cluster prevalence interpretable as a global mixing proportion.

The fitted object is subsequently summarized through a representative partition together with cluster-specific response summaries.

```
fit_clust <- dpmgpd.cluster(
  formula    = crim ~ lstat + rm + nox,
  data       = train_dat,
  kernel     = "lognormal",
  type       = "weights",
  components = 15,
  mcmc       = mcmc_fixed
)
```

4.4. Representative partition and cluster-specific response summaries

Because mixture components are not identifiable up to permutation, the training labels are still extracted from the posterior through the PSM and Dahl's least-squares representative partition. For presentation, however, we focus on the resulting partition and the response distributions within its representative clusters rather than on the full PSM heatmap.

The following command extracts the representative labels for the training sample.

```
z_train <- predict(fit_clust, type = "label")
```

For presentation we summarize the first five most populated clusters from `summary(fit_clust,top_n=5)` and report the associated descriptive statistics in Table 2.

4.5. Cluster labels for new observations

Cluster assignment for new observations is performed through the same prediction interface. The output is a label in the space of the representative training clusters, obtained by aggregating posterior predictive co-clustering evidence at the cluster level.

```
z_test <- predict(
  fit_clust,
  newdata = test_dat,
```

```
  type     = "label"
)
```

For the test sample we tabulate only the predicted cluster sizes that are actually represented in the new observations (Table 1).

Table 1: Test cluster sizes

cluster	n
C1	127

To summarize the dominant representative clusters, Table 2 reports descriptive statistics for up to the five most populated training clusters. For each cluster we show the cluster size together with within-cluster means and standard deviations for the response and standardized covariates.

Table 2: Descriptive summaries for up to the five most populated representative clusters

cluster	n	crim_mean	crim_sd	lstat_mean	lstat_sd	rm_mean	rm_sd	nox_mean	nox_sd	certainty_mean	certainty_sd
C1	327	3.86	8.772	-0.064	0.991	0.064	1.027	-0.107	0.905	0.601	0.020
C2	52	3.45	12.453	0.403	0.973	-0.403	0.692	0.673	1.280	0.594	0.074

Figure 3 shows response boxplots for the test sample only, restricted to the five most populated predicted clusters. Clusters with no test observations are omitted automatically.

```
cluster_counts <- sort(table(z_test$labels), decreasing = TRUE)
top_clusters <- as.integer(names(cluster_counts)[seq_len(min(5L, length(cluster_counts)))]

plot_df <- data.frame(
  cluster = z_test$labels,
  response = z_test$data$crim
)
plot_df <- subset(plot_df, cluster %in% top_clusters)

cluster_levels <- names(cluster_counts)[names(cluster_counts) %in% as.character(top_cluste
plot_df$cluster <- factor(paste0("C", plot_df$cluster), levels = paste0("C", cluster_level

ggplot(plot_df, aes(x = cluster, y = response)) +
  geom_boxplot(
    width = 0.65,
    staplewidth = 0.45,
    outlier.alpha = 0.55,
    fill = "#b3d7ff",
    color = "#1f3b6d"
  ) +
  labs(x = "Predicted cluster", y = "Response (crim)") +
  theme_minimal(base_size = 11)
```

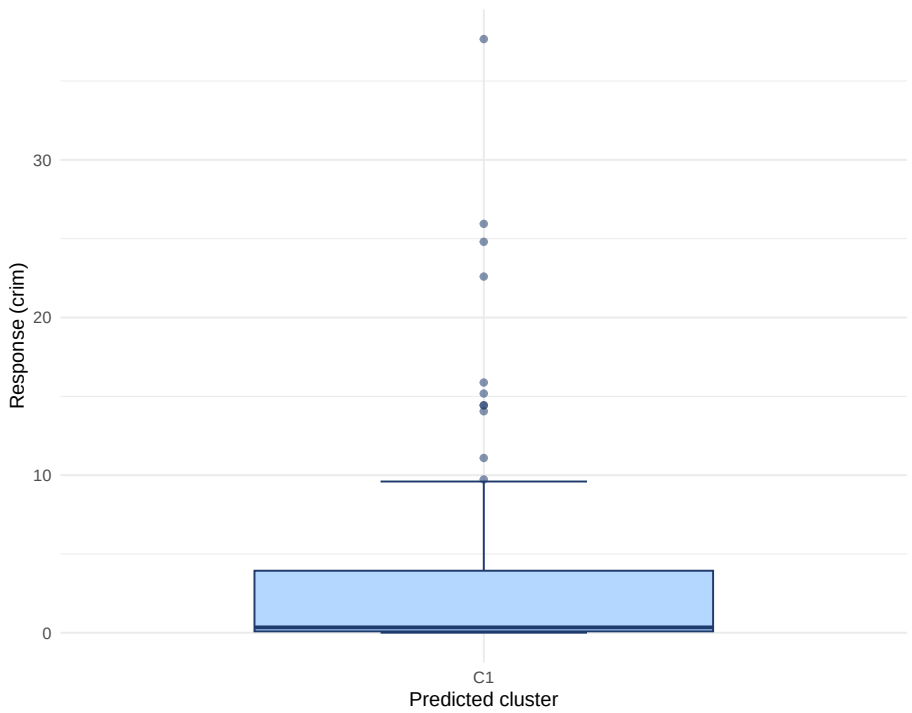

Figure 3: Response boxplots for the five most populated predicted test clusters.

This analysis illustrates the intended clustering workflow: inference is summarized through a representative training partition and its cluster-level descriptive summaries, and new observations are assigned to those clusters using posterior predictive co-clustering evidence. In Section 5, we next turn to the causal workflow, which targets treatment contrasts rather than latent partitions.

5. Data analysis II: Causal Inference

This application demonstrates causal estimands from a two-arm spliced DPM regression with a GPD tail. In contrast to Section 4, where the focus is model-based clustering under a CRP bulk-only regression, here the focus is inference on restricted-mean and quantile treatment contrasts, including conditional (covariate-specific) effects and extreme upper-tail behavior.

5.1. Dataset and objective

The AER package does not ship the National Job Training Partnership Act (JTPA) experimental data as an in-package dataset. Instead, we use a bundled causal benchmark data object to provide a clean and reproducible entry point for a heavy-tailed earnings-style application.

Let Y denote post-program earnings (USD), $A \in \{0, 1\}$ denote randomized assignment to training, and $\boldsymbol{x}$ denote baseline covariates including previous earnings and demographics. The scientific goal is to quantify how training changes (i) upper-tail-truncated average earnings and (ii) distributional features of earnings, with special attention to the upper tail.

For this heavy-tailed benchmark, unrestricted mean effects are sensitive to a small number of extreme posterior draws. I therefore use restricted-mean versions of the mean-based causal estimands with cutoff c equal to the empirical 95th percentile of observed earnings. Throughout, we report four causal estimands: restricted mean average treatment effect (RMATE), quantile treatment effect (QTE), restricted mean conditional average treatment effect (RMCATE), and conditional quantile treatment effect (CQTE).

$$\begin{aligned}
\mathrm{RMATE}(c) &= \mathbb{E}\big[\min\{Y(1), c\}\big] - \mathbb{E}\big[\min\{Y(0), c\}\big], \\
\mathrm{QTE}(\tau) &= Q_{Y(1)}(\tau) - Q_{Y(0)}(\tau), \\
\mathrm{RMCATE}(c, \boldsymbol{x}) &= \mathbb{E}\big[\min\{Y(1), c\} \mid \boldsymbol{x}\big] \\
&\quad - \mathbb{E}\big[\min\{Y(0), c\} \mid \boldsymbol{x}\big], \\
\mathrm{CQTE}(\tau \mid \boldsymbol{x}) &= Q_1(\tau \mid \boldsymbol{x}) - Q_0(\tau \mid \boldsymbol{x}).
\end{aligned}$$

Here $Q_a(\tau \mid \boldsymbol{x})$ denotes the arm-specific τ-quantile. The standardized estimands (RMATE, QTE) integrate over the empirical distribution of $\{\boldsymbol{x}_i\}_{i=1}^n$; the conditional estimands (RMCATE, CQTE) are evaluated at user-chosen covariate profiles.

5.2. Spliced DPM regression model

I model arm-specific conditional outcome distributions $F_a(\cdot \mid \boldsymbol{x})$ via a spliced construction:

$$f_a(y \mid \boldsymbol{x}) = \sum_j w_{aj}(\boldsymbol{x})\, k(y \mid \Theta_{aj})\, \mathbf{1}(y \le u) + \{1 - F_a(u \mid \boldsymbol{x})\}\, g(y \mid u, \sigma_a, \xi_a)\, \mathbf{1}(y > u),$$

where the bulk kernel $k(\cdot)$ is a DPM regression (stick-breaking backend) and the tail density $g(\cdot)$ is GPD with scale $\sigma_a > 0$ and shape ξ_a. The GPD tail index ξ_a controls extremal heaviness (e.g., $\xi_a > 0$ corresponds to heavy tails). This model stabilizes inference for extreme quantiles by coupling nonparametric bulk flexibility with EVT-consistent tail extrapolation.

5.3. Model fitting

I fit a two-arm spliced mixture using a lognormal bulk kernel. The MCMC settings below are intentionally modest for vignette speed; in substantive analysis we recommend longer runs and sensitivity checks (e.g., components, threshold priors, and prior scales).

```
fit <- dpmgpd.causal(
  x = y,
  X = X,
  treat = A,
  backend = "sb",
  kernel = "lognormal",
  components = 4,
  mcmc = c(mcmc_fixed, list(show_progress = FALSE))
)
```

5.4. Standardized estimands: restricted mean and QTE

I first summarize the overall effect using a restricted-mean ATE, implemented by `ate(type = "rmean", cutoff = c)` with $c =$ ‘$rround(rm_c utoff, 3)$‘. This keeps the mean-based contrast

finite and interpretable on a common earnings scale. I then compute QTE on a quantile grid that extends into the upper tail but stops short of the most unstable 0.99 level in the main text.

```
ate_fit <- ate(
  fit,
  type = "rmean",
  cutoff = rm_cutoff,
  show_progress = FALSE
)
```

Table 3: Restricted-mean ATE with cutoff c = 8.036

id	estimate	lower	upper
1	0.816	0.417	1.199

```
qte_fit <- qte(fit, probs = taus, show_progress = FALSE)
```

Table 4: Marginal QTE estimates across the main-text quantile grid

id	index	estimate	lower	upper
1	0.25	-0.161	-0.320	-0.001
1	0.50	0.674	-0.135	1.659
1	0.75	1.561	0.683	2.564
1	0.90	4.702	1.951	9.707
1	0.95	11.531	4.315	29.668

```
ate_effect_plot <- plot(ate_fit, type = "effect")
ate_effect_plot
```

```
qte_effect_plot <- plot(qte_fit, type = "effect")
qte_effect_plot
```

The marginal treatment-effect visual summaries are shown in Figure 4, captioned "Posterior restricted-mean ATE summary for the spliced causal model with cutoff $c =$ '$rround(rm_cutoff, 3)$'," for the restricted-mean ATE, and Figure 5, captioned "Posterior QTE summary across the main-text quantile grid." for the QTE.

5.5. Conditional estimands: restricted mean CATE and CQTE at representative profiles

To illustrate heterogeneity, we evaluate conditional effects at two covariate profiles: (i) a "typical" profile at sample medians and (ii) a "high previous earnings" profile at the 80th percentile of previous earnings, holding other covariates at medians.

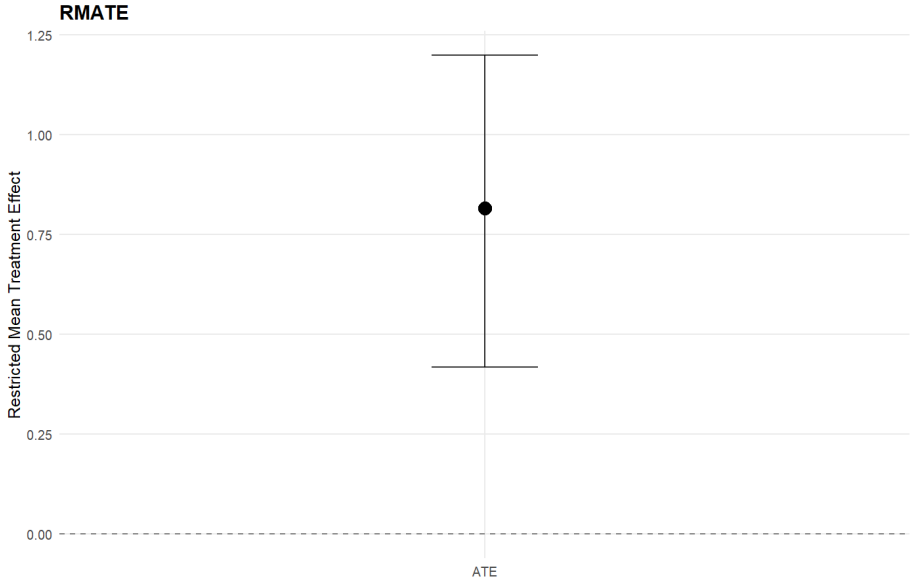

Figure 4: Posterior restricted-mean ATE summary for the spliced causal model with cutoff $c = 8.036$.

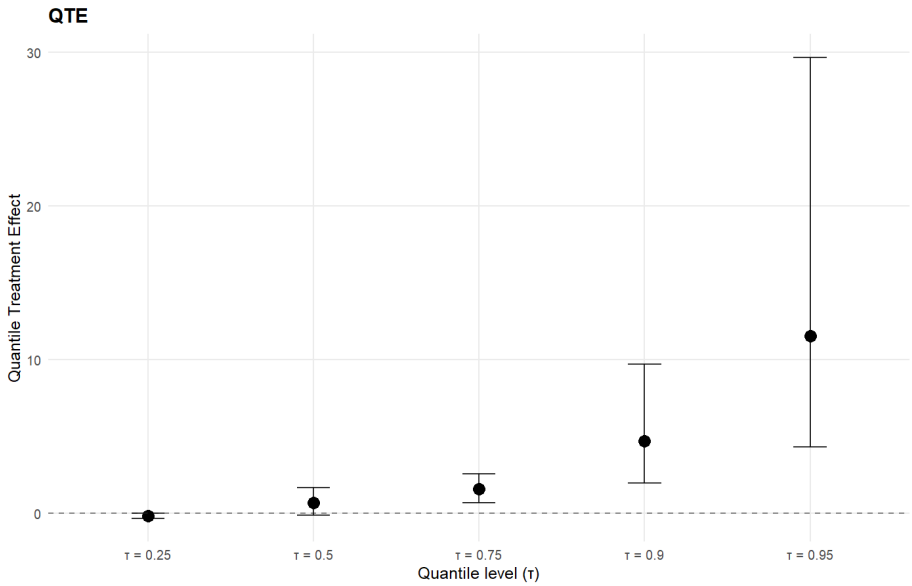

Figure 5: Posterior QTE summary across the main-text quantile grid.

```
col_med <- apply(X, 2, median, na.rm = TRUE)
x_typical <- col_med
x_highprev <- col_med
profile_var <- if ("prevearn" %in% colnames(X)) "prevearn" else colnames(X)[1]
x_highprev[profile_var] <- as.numeric(quantile(X[, profile_var], 0.80, na.rm = TRUE))
xnew <- rbind(typical = x_typical, high_prev_earn = x_highprev)
```

Table 5: Representative profiles used for conditional causal contrasts; the high-profile shift is applied to x1.

	x1	x2	x3
typical	0.076	-0.096	0.012
high_prev_earn	0.797	-0.096	0.012

```
cate_fit <- cate(
  fit,
  newdata = xnew,
  type = "rmean",
  cutoff = rm_cutoff,
  show_progress = FALSE
)
```

Table 6: Restricted-mean CATE estimates with cutoff c = 8.036

id	estimate	lower	upper
1	0.825	0.239	1.348
2	0.950	0.332	1.499

```
cqte_fit <- cqte(fit, probs = taus, newdata = xnew, show_progress = FALSE)
```

Table 7: Conditional QTE estimates at the representative profiles

id	index	estimate	lower	upper
1	0.25	-0.202	-0.317	-0.089
1	0.50	0.000	0.000	0.000
1	0.75	1.059	0.535	1.612
1	0.90	1.299	0.758	1.786
1	0.95	1.664	1.109	2.268
2	0.25	-0.202	-0.317	-0.089
2	0.50	0.000	0.000	0.000
2	0.75	1.074	0.568	1.637
2	0.90	1.684	1.071	2.227
2	0.95	2.481	1.535	3.350

```
cate_effect_plot <- plot(cate_fit, type = "effect")
cate_effect_plot
```

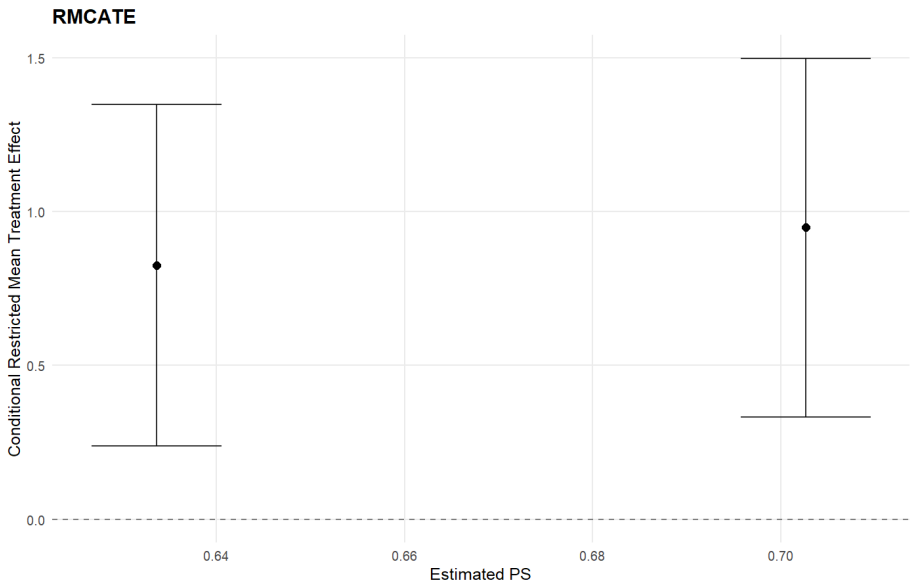

Figure 6: Posterior restricted-mean CATE summary at the representative profiles, with cutoff $c = 8.036$.

```
cqte_effect_plot <- plot(cqte_fit, type = "effect", facet_by = "id")
cqte_effect_plot
```

The conditional effect plots are shown in Figure 6 for the restricted-mean CATE and Figure 7 for the CQTE profiles.

5.6. Tail behavior

A key advantage of the spliced model is direct posterior inference on the tail index ξ_a by arm. Larger ξ_a indicates a heavier upper tail and greater propensity for extreme earnings.

Table 8: Posterior summaries of the arm-specific GPD tail-shape parameters

arm	mean	sd	q0.025	q0.500	q0.975
treated	0.557	0.053	0.464	0.556	0.646
control	0.414	0.079	0.319	0.407	0.563

5.7. Interpretation

The restricted-mean ATE indicates a positive average earnings gain below the empirical cutoff, and the QTE profile shows that the treatment effect strengthens markedly in the upper part of the outcome distribution. The restricted-mean CATE estimates are positive at both

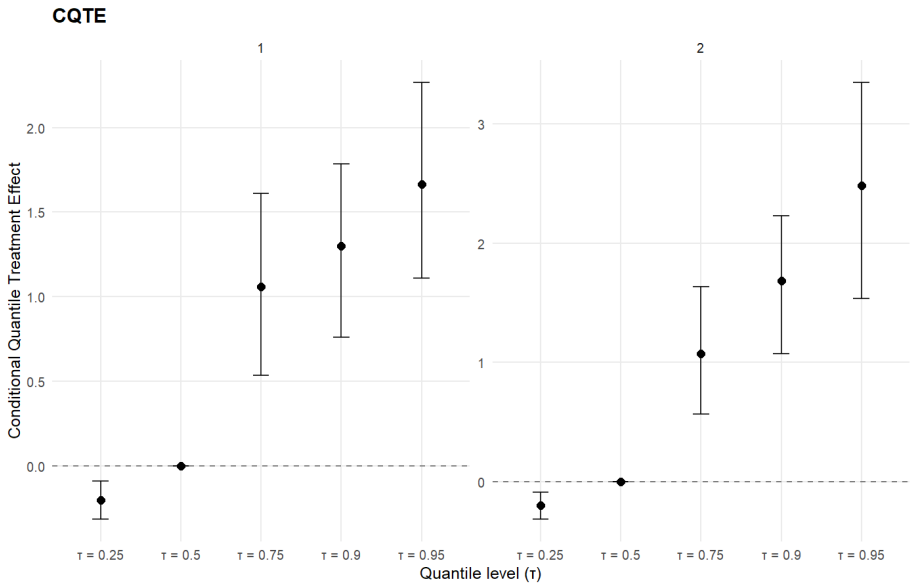

Figure 7: Posterior CQTE summaries at the representative profiles.

representative profiles, while the CQTE curves show stronger upper-tail gains for the higher previous-earnings profile. Finally, the treated arm has the larger posterior tail index ξ_a, indicating a heavier upper tail under training and explaining why unrestricted mean contrasts are much less stable in this application.

6. Discussion

CausalMixGPD was designed to connect three tasks that are often handled with separate tools: flexible modeling of the central part of a response distribution, principled extrapolation of upper-tail behavior, and causal comparison of complete outcome distributions. The package does this by keeping the bulk model nonparametric, introducing generalized Pareto structure only where tail extrapolation is needed, and exposing posterior summaries through a common workflow. In particular, the same fitted object can support unconditional or conditional density estimation, tail-aware prediction, clustering summaries, and causal effect estimation based on treatment-specific predictive distributions.

A practical advantage of this strategy is that inference is organized around the posterior predictive distribution rather than around a single summary such as the mean. Once a model is fitted, densities, survival curves, quantiles, means, restricted means, and tail summaries are all derived from the same posterior object. In the causal setting, this is especially useful because it allows average effects, quantile treatment effects, and tail contrasts to be reported within a single coherent model rather than through disconnected estimators. Modeling the full distribution also helps avoid internally inconsistent summaries, including quantile-crossing issues that can arise when separate quantiles are estimated independently across probability levels.

The article emphasized the main semiparametric construction, but the software contribution is intentionally broader. The package supports both stick-breaking and Chinese restaurant

process representations, multiple kernel families for real-line and positive-support outcomes, optional GPD splicing, arm-specific modeling choices in causal analyses, and a clustering workflow built from the same fitted-mixture machinery rather than from a separate ad hoc module. In the clustering setting, predictor dependence can enter through weights, atoms, or both, matching common dependent-DP formulations for regression clustering (MacEachern 1999, 2000; De Iorio *et al.* 2004; Quintana *et al.* 2022). Posterior partition uncertainty is reported through posterior similarity matrices and Dahl representative partitions (Binder 1978; Dahl 2006), which makes the clustering output interpretable even when component labels switch across MCMC draws. This breadth is useful in applications where the same dataset motivates more than one inferential target: exploratory density estimation, subgroup discovery, prediction at extreme quantiles, and distributional treatment-effect reporting can all be pursued without switching to unrelated software.

This clustering contribution matters substantively, not just computationally. In many applications the goal is not only to estimate an overall density, but also to identify subpopulations with distinct regression surfaces, different bulk shapes, or differing exposure to extreme outcomes. By keeping clustering, bulk-tail modeling, and posterior prediction inside one workflow, CausalMixGPD allows users to move from exploratory subgroup discovery to tail-risk assessment and, when relevant, causal reporting without changing the underlying model class.

These strengths should be weighed against several limitations. First, the methodology is computationally intensive. Posterior simulation through nimble is flexible and transparent, but MCMC for mixture models with covariate-dependent parameters and optional tail components can be expensive, especially when multiple chains are used or when the causal extension fits separate arm-specific outcome models in addition to a propensity score stage. As sample size, covariate dimension, or the truncation level increases, routine sensitivity analysis and model comparison become more costly.

Second, the spliced model inherits the main tuning decisions of both Dirichlet process mixtures and peaks-over-threshold modeling. Results may depend on the kernel family, truncation level, prior specification, and the threshold/tail parameterization. EVT provides asymptotic support for GPD modeling above high thresholds, but finite-sample tail estimation still depends on having enough exceedances to identify tail parameters well. For this reason, tail summaries should be interpreted alongside posterior predictive checks, sensitivity analyses, and substantive knowledge about the outcome scale rather than as automatic black-box outputs. The same caution applies to clustering in spliced models: when groups overlap strongly, predictor effects are weak, or the number of exceedances is small, the posterior similarity matrix may reveal substantial partition uncertainty and should often be treated as more informative than any single hard clustering.

Third, the current causal implementation is deliberately focused. It is built for binary treatments and continuous outcomes under standard identification assumptions such as consistency, ignorability, and overlap. Propensity score augmentation can improve adjustment in observational studies, but it does not replace careful design, diagnostics for limited overlap, or sensitivity analysis to unmeasured confounding. Likewise, the present implementation is not yet a dedicated framework for censored survival outcomes, multivariate responses, longitudinal treatment regimes, or interference between units.

These limitations point to several concrete directions for future work. Methodologically,

more data-adaptive threshold selection, richer tail regression structures, and faster approximate posterior computation could improve robustness and scalability. On the causal side, extensions to multivalued or continuous treatments, stronger overlap diagnostics, and doubly robust distributional estimands would broaden the range of applications. On the clustering side, scalable posterior similarity matrix approximations, alternative partition-loss summaries, and cluster-specific tail diagnostics would broaden applicability in larger or more weakly separated datasets. From a software perspective, the registry-based package architecture makes it natural to add new kernels, tail components, and advanced extensions. We therefore view CausalMixGPD not only as an applied package for tail-aware density estimation, clustering, and causal inference, but also as a foundation for further work on Bayesian distributional modeling in R.

References

(2023). *BCEE: The Bayesian Causal Effect Estimation Algorithm*. doi:10.32614/CRAN.package.BCEE. R package on CRAN, URL https://cran.r-project.org/package=BCEE.

(2023). *dirichletprocess: Build Dirichlet Process Objects for Bayesian Modelling*. doi:10.32614/CRAN.package.dirichletprocess. R package on CRAN, URL https://cran.r-project.org/package=dirichletprocess.

(2024). *bcf: Causal Inference using Bayesian Causal Forests*. doi:10.32614/CRAN.package.bcf. R package on CRAN, URL https://cran.r-project.org/package=bcf.

(2024). *POT: Generalized Pareto Distribution and Peaks Over Threshold*. doi:10.32614/CRAN.package.POT. R package on CRAN, URL https://cran.r-project.org/package=POT.

(2025). *bartCause: Causal Inference using Bayesian Additive Regression Trees*. doi:10.32614/CRAN.package.bartCause. R package on CRAN, URL https://cran.r-project.org/package=bartCause.

(2025). *evdbayes: Bayesian Analysis in Extreme Value Theory*. doi:10.32614/CRAN.package.evdbayes. R package on CRAN, URL https://cran.r-project.org/package=evdbayes.

(2025). *ggmcmc: Analysis of MCMC Samples and Bayesian Inference*. doi:10.32614/CRAN.package.ggmcmc. R package on CRAN, URL https://cran.r-project.org/package=ggmcmc.

(2025). *nimble: MCMC, Particle Filtering, and Programmable Hierarchical Modeling*. doi:10.32614/CRAN.package.nimble. R package on CRAN, URL https://cran.r-project.org/package=nimble.

Balkema AA, de Haan L (1974a). "Residual Life Time at Great Age." *Annals of Probability*, **2**(5), 792–804. doi:10.1214/aop/1176996548.

Balkema AA, de Haan L (1974b). "Residual Life Time at Great Age." *The Annals of Probability*, **2**(5), 792–804. doi:10.1214/aop/1176996548.

Behrens CN, Lopes HF, Gamerman D (2004). "Bayesian Analysis of Extreme Events with Threshold Estimation." *Statistical Modelling*, **4**(3), 227–244. doi:10.1191/1471082X04st075oa.

Binder DA (1978). "Bayesian Cluster Analysis." *Biometrika*, **65**(1), 31–38. doi:10.1093/biomet/65.1.31.

Blackwell D, MacQueen JB (1973). "Ferguson Distributions Via Pólya Urn Schemes." *The Annals of Statistics*, **1**(2), 353–355. doi:10.1214/aos/1176342372.

Coles S (2001). *An Introduction to Statistical Modeling of Extreme Values*. Springer, London. doi:10.1007/978-1-4471-3675-0.

Corradin R, Canale A, Nipoti B (2021). "BNPmix: An R Package for Bayesian Nonparametric Modeling via Pitman–Yor Mixtures." *Journal of Statistical Software*, **100**(15), 1–47. doi:10.18637/jss.v100.i15.

Dahl DB (2006). *Model-Based Clustering for Expression Data via a Dirichlet Process Mixture Model*. Cambridge University Press. In: *Bayesian Inference for Gene Expression and Proteomics*, Kim-Anh Do, Peter Müller, Marina Vannucci (Eds.).

de Haan L, Ferreira A (2007). *Extreme Value Theory: An Introduction*. Springer Science & Business Media.

De Iorio M, Müller P, Rosner GL, MacEachern SN (2004). "An ANOVA Model for Dependent Random Measures." *Journal of the American Statistical Association*, **99**(465), 205–215. doi:10.1198/016214504000000205.

de Valpine P, Turek D, Paciorek CJ, Anderson-Bergman C, Temple Lang D, Bodik R (2017). "Programming with Models: Writing Statistical Algorithms for General Model Structures with NIMBLE." *Journal of Statistical Software*, **76**(10), 1–27. doi:10.18637/jss.v076.i10.

Durrieu G, Grama I, Jaunatre K, Pham QK, Tricot JM (2018). "extremefit: A Package for Extreme Quantiles." *Journal of Statistical Software*, **87**(12), 1–20. doi:10.18637/jss.v087.i12.

Escobar MD, West M (1995). "Bayesian Density Estimation and Inference Using Mixtures." *Journal of the American Statistical Association*, **90**(430), 577–588. doi:10.1080/01621459.1995.10476550.

Ferguson TS (1973). "A Bayesian Analysis of Some Nonparametric Problems." *The Annals of Statistics*, **1**(2), 209–230. doi:10.1214/aos/1176342360.

Firpo S (2007). "Efficient Semiparametric Estimation of Quantile Treatment Effects." *Econometrica*, **75**(1), 259–276. doi:10.1111/j.1468-0262.2007.00738.x.

Frigessi A, Haug O, Rue H (2002). "A Dynamic Mixture Model for Unsupervised Tail Estimation without Threshold Selection." *Extremes*, **5**(3), 219–235. doi:10.1023/A:1024072610684.

Gelman A, Carlin JB, Stern HS, Dunson DB, Vehtari A, Rubin DB (2013). *Bayesian Data Analysis*. 3 edition. Chapman and Hall/CRC, Boca Raton, FL. doi:10.1201/b16018.

Hill JL (2011). "Bayesian Nonparametric Modeling for Causal Inference." *Journal of Computational and Graphical Statistics*, **20**(1), 217–240. doi:10.1198/jcgs.2010.08162.

Imbens GW, Rubin DB (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press, Cambridge.

Jara A, Hanson T, Quintana F, Müller P, Rosner G (2011). "DPpackage: Bayesian Semi- and Nonparametric Modeling in R." *Journal of Statistical Software*, **40**(5), 1–30. doi:10.18637/jss.v040.i05.

Lo AY (1984). "On a Class of Bayesian Nonparametric Estimates: I. Density Estimates." *The Annals of Statistics*, **12**(1), 351–357. doi:10.1214/aos/1176346412.

MacEachern SN (1999). *Dependent Nonparametric Processes*. ASA Proceedings of the Section on Bayesian Statistical Science, URL https://u.osu.edu/maceachern.1/files/2025/05/1999-MacEachern-JSM-Proceedings.pdf.

MacEachern SN (2000). *Dependent Dirichlet Processes*. Technical Report, Department of Statistics, The Ohio State University, URL https://u.osu.edu/maceachern.1/files/2025/05/2000-MacEachern-DDP-Tech-Report.pdf.

Neal RM (2000). "Markov Chain Sampling Methods for Dirichlet Process Mixture Models." *Journal of Computational and Graphical Statistics*, **9**(2), 249–265. doi:10.1080/10618600.2000.10474879.

Neyman J (1990). "On the Application of Probability Theory to Agricultural Experiments. Essay on Principles. Section 9." *Statistical Science*, **5**(4), 465–480.

Pickands J (1975). "Statistical Inference Using Extreme Order Statistics." *The Annals of Statistics*, **3**(1), 119–131. doi:10.1214/aos/1176343003.

Plummer M, Best N, Cowles K, Vines K (2024). *coda: Output Analysis and Diagnostics for MCMC*. R package version 0.19-4.1, URL https://cran.r-project.org/package=coda.

Quintana FA, Müller P, Jara A, MacEachern SN (2022). "The Dependent Dirichlet Process and Related Models." *Statistical Science*, **37**(1), 24–49. doi:10.1214/20-STS819.

Ren L, Du L, Carin L, Dunson DB (2011). "The Logistic Stick-Breaking Process." *Journal of Machine Learning Research*, **12**(7), 203–239. URL https://www.jmlr.org/papers/v12/ren11a.html.

Rosenbaum PR, Rubin DB (1983). "The Central Role of the Propensity Score in Observational Studies for Causal Effects." *Biometrika*, **70**(1), 41–55. doi:10.1093/biomet/70.1.41.

Rubin DB (1974). "Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies." *Journal of Educational Psychology*, **66**(5), 688–701. doi:10.1037/h0037350.

Sethuraman J (1994). "A Constructive Definition of Dirichlet Priors." *Statistica Sinica*, **4**(2), 639–650.

Talbot D, Lefebvre G, Atherton J, Chiu Y (2015). "The Bayesian Causal Effect Estimation Algorithm." *Journal of Causal Inference*, **3**(2), 207–236. doi:10.1515/jci-2014-0035.

7. Supplementary Materials

This supplement replaces the earlier narrative list with a compact reference for the package's customization surface. The emphasis is on available modeling switches, arm- and task-specific overrides, and the advanced extension points most relevant for extending CausalMixGPD.

User-facing workflow map

Build layer: `bundle()` or task-specific wrappers choose `kernel`, `backend`, `GPD`, and `components`.

⇓

Specification layer: optional overrides through `param_specs`, propensity-score augmentation, and clustering dependence settings.

⇓

Run layer: `mcmc()`, `run_mcmc_bundle_manual()`, or `run_mcmc_causal()` compile and sample the selected model.

⇓

Output layer: a common fit object feeds posterior summaries, prediction, causal effects, tails, and clustering diagnostics.

Figure 8: Detailed user-facing workflow for model build, optional override, run, and summary extraction. The stages correspond to the package entry points listed in Table 9 and are written to match exported function names.

7.1. Core customization map

Interface	Scope	Main choices	Notes
`bundle()`	Build only	raw vector versus formula input; optional `treat`; `GPD`	Chooses between one-arm and causal bundles without running MCMC, which makes it the natural entry point for inspect-then-run workflows.
`dpmix()` and `dpmgpd()`	One-arm fit	`kernel`; `backend`; `components`; `param_specs`; `mcmc`	Convenience wrappers for bulk-only and spliced fits. They share the same bundle architecture and differ mainly in whether the GPD tail is active.
`dpmix.causal()` and `dpmgpd.causal()`	Two-arm causal fit	arm-specific `kernel`, `backend`, `GPD`, `components`; propensity-score controls	Wrap paired outcome models and an optional PS stage, then expose causal effect summaries from the resulting arm-specific predictive distributions.
`dpmix.cluster()` and `dpmgpd.cluster()`	Clustering fit	`type`; `link`; `priors`; `components`; `mcmc`	Formula-driven clustering interface with label-invariant summaries, posterior similarity matrices, and representative partitions.
`mcmc()`	Run only	stored bundle plus MCMC overrides	Keeps the build/run split explicit by dispatching to the correct runner after the bundle has been inspected or programmatically edited.

Table 9: Main user-facing entry points and where customization occurs in the workflow. Rows map directly to exported wrappers, builders, and runners in the package application programming interface (API) and separate build-time versus run-time controls.

oprule Workflow/interface	Default choice	Where set	How to customize
`dpmix()` and `dpmix.causal()`	`GPD=FALSE`	wrapper contract	Switch to `dpmgpd()` or `dpmgpd.causal()`, or build a bundle with `GPD=TRUE`.
`dpmgpd()` and `dpmgpd.causal()`	`GPD=TRUE`	wrapper contract	Use the corresponding `dpmix*` wrapper, or pass a bundle with tail disabled.
One-arm and causal builders	`backend="sb"`; `components=n`; `monitor="core"`; `epsilon=0.025`; `alpha_random=TRUE`	`build_nimble_bundle()` and `build_causal_bundle()` defaults	Override directly in `bundle(...)` or in the explicit builder calls.
Causal-only options	`PS="logit"`; `ps_scale="logit"`; `ps_summary="mean"`	`build_causal_bundle()` defaults	Set `PS`, `ps_scale`, `ps_summary`, and related `ps_*` controls.
Clustering wrappers	`type="weights"`; `default="weights"`; `GPD` selected by wrapper family	`dpmix.cluster()` and `dpmgpd.cluster()` defaults	Choose `type="param"` or `"both"`, and use the GPD or non-GPD wrapper accordingly.

Table 10: Default choices across the main customization interfaces, with where those defaults are defined and the primary override path. This table is intended as a quick start map before using the detailed controls in Tables 11–14.

Option	Accepted values	Applies to	Interpretation
`backend`	`"sb"`, `"crp"`, `"spliced"`	one-arm and causal bundles	`"sb"` uses finite stick-breaking truncation; `"crp"` uses latent cluster labels; `"spliced"` requests explicit bulk-tail dispatch and reverts to CRP behavior when `GPD=FALSE`.
`kernel`	`normal`, `lognormal`, `invgauss`, `gamma`, `laplace`, `amoroso`, `cauchy`	all workflows	Selects the bulk distribution family. Causal fits may use different kernels across treatment arms. Cluster fits draw from the same kernel registry as density and causal fits.
`GPD`	`TRUE` or `FALSE`	all workflows	Activates threshold and tail parameters when supported by the chosen kernel. The Cauchy kernel is intentionally bulk-only.
`components`	integer ≥ 2	all workflows	Single truncation control: SB interprets it as mixture truncation, whereas CRP treats it as the maximum represented number of clusters. Causal fits may use different values by arm.
`alpha_random`	`TRUE` or `FALSE`	bundle and cluster builders	Controls whether the DP concentration parameter is stochastic under its default prior or fixed through a user-supplied specification.
`monitor`	`"core"` or `"full"`	all builders	`"full"` automatically activates broader monitoring, including latent allocations and stick-breaking terms when applicable.
`monitor_latent`, `monitor_v`	`TRUE` or `FALSE`	all builders and runners	Allow fine control over whether latent cluster labels and SB stick weights are stored in the retained posterior output.
`epsilon`	numeric in $[0, 1)$	all builders	Used in downstream truncation summaries, plots, and renormalized component filtering. Smaller values retain a larger fraction of the mixture support in posterior summaries.
`mcmc`, `mcmc_outcome`, `mcmc_ps`	`niter`, `nburnin`, `thin`, `nchains`, `seed`	all workflows	The core iteration controls. Causal fits separate outcome-stage and propensity-score-stage settings so that the two stages can be tuned independently.

Table 11: Core modeling controls reused across one-arm, causal, and clustering workflows. Accepted values are aligned with registry constants and argument validation in the build/run and causal interfaces.

Kernel	Support	Bulk parameters	Default covariate-linked bulk parameters	GPD
`normal`	$\mathbb{R}$	`mean`, `sd`	`mean` with identity link	Yes
`lognormal`	$\mathbb{R}_+$	`meanlog`, `sdlog`	`meanlog` with identity link	Yes
`invgauss`	$\mathbb{R}_+$	`mean`, `shape`	`mean` with exponential link	Yes
`gamma`	$\mathbb{R}_+$	`shape`, `scale`	`scale` with exponential link	Yes
`laplace`	$\mathbb{R}$	`location`, `scale`	`location` with identity link	Yes
`amoroso`	$\mathbb{R}_+$	`loc`, `scale`, `shape1`, `shape2`	`loc` with identity link; `scale` with exponential link; `shape1` fixed at 1 by default	Yes
`cauchy`	$\mathbb{R}$	`location`, `scale`	`location` with identity link	No

Table 12: Kernel families currently available through the shared registry, including support, parameterization, default covariate-linked bulk parameters, and GPD-tail compatibility. Entries correspond to kernel metadata defined in `get_kernel_registry()`.

Option	Values	Effect on causal workflow
`backend`, `kernel`, `GPD`, `components`, `epsilon`	length 1 or length 2 values	A single value is shared across treated and control arms; length-2 input lets the treated and control outcome models use different backends, kernels, tail usage, truncation levels, and posterior truncation thresholds.
`param_specs`	single list or `list(con=..., trt=...)`	Supports arm-specific bulk and tail priors, fixed values, and link structures while preserving the same high-level causal summary API.
`PS`	`"logit"` (default), `"probit"`, `"naive"`, `FALSE`	Turns the propensity-score stage on or off and chooses the PS model family used before arm-specific outcome fitting.
`ps_scale`	`"logit"` or `"prob"`	Controls whether the PS summary enters the outcome model on the probability scale or the logit scale.
`ps_summary`	`"mean"` or `"median"`	Determines which posterior summary of the PS is appended to the outcome covariate vector.
`ps_prior`, `include_intercept`, `ps_clamp`	prior list; logical; positive numeric	Tune prior strength in the PS model, whether an intercept is prepended, and how aggressively extreme fitted PS values are clamped away from 0 and 1.
`mcmc_outcome`, `mcmc_ps`	stage-specific MCMC lists	Allow the PS stage and the two outcome stages to use different iteration budgets and chain settings.
`parallel_arms`, `workers`, `z_update_every`	runner controls	`run_mcmc_causal()` exposes arm-level execution controls and can thin latent-label updates when cluster-allocation moves dominate cost; parallel requests may fall back to sequential execution depending on runtime availability.

Table 13: Causal-model customization options beyond the one-arm baseline, including arm-specific overrides and propensity-score augmentation controls. Values and semantics follow the causal builder and runner argument contracts.

Option	Values	Effect on clustering workflow
`type`	`"weights"`, `"param"`, `"both"`	Determines whether predictors influence mixture weights, kernel parameters, or both. This is the main modeling choice for predictor-dependent clustering.
Implicit backend choice	`"param"` → CRP; `"weights"` or `"both"` → SB	The clustering interface chooses the backend automatically to match the requested dependence structure, so the user typically reasons in terms of clustering mode rather than backend internals.
`link`	named overrides for bulk and GPD parameters	Refines which parameters are regressed on covariates and which inverse links are used, without having to rewrite the kernel registry defaults.
`priors`	named prior overrides	Adjusts cluster-gating or parameter priors for the clustering fit while reusing the same code-generation pipeline as the density models.
`components`	integer ≥ 2	If omitted, the builder defaults to $\max(20, \min(50, n))$, which is deliberately more conservative than the one-arm wrappers because clustering performance is sensitive to under-truncation.
`monitor`, `monitor_latent`, `monitor_v`	monitor profile and logical flags	Latent labels are always tracked for clustering summaries, and SB stick weights can be monitored explicitly when the chosen dependence mode uses an SB representation.

Table 14: Customization options specific to the clustering interface, including dependence mode, backend implication, and monitoring behavior. Defaults and constraints reflect clustering builders and internal backend dispatch rules.

7.2. Advanced customization

The most important low-level knob for power users is `param_specs`, which lets individual parameters move between fixed, prior-driven, and covariate-linked representations. In current code generation, the supported inverse-link names include `identity`, `exp`, `softplus`, `log`, and `power`; the package defaults shown in Table 12 are simply the starting point.

Mode	Representative structure	Interpretation	Typical use
`fixed`	`list(mode="fixed",value=...)`	Locks the parameter to a constant rather than sampling it from a prior or regressing it on covariates.	Holding a shape parameter fixed, enforcing a known threshold, or reproducing a simpler submodel.
`dist`	`list(mode="dist",dist=...,args=list(...))`	Treats the parameter as random under a prior but without regression on covariates.	Default component-specific bulk parameters and scalar tail-shape parameters.
`link`	`list(mode="link",link=...,beta_prior=...)`	Builds a regression layer for the parameter and applies the specified inverse link to the linear predictor.	Covariate-dependent bulk location/scale parameters, PS-adjusted outcome regression, or linked tail-scale models.
`link+dist`	`threshold=list(mode="link",link=...,link_dist=list(dist="lognormal"))`	Combines a regression mean structure with additional stochastic dispersion around the linked value.	Observation-specific thresholds with extra uncertainty, which is the default spliced threshold pattern in several covariate-dependent fits.

Table 15: Parameter-plan modes available through `param_specs`, showing how fixed constants, prior-only parameters, and covariate-linked parameters are encoded in compiled model plans.

7.3. Advanced core extension points

The package architecture is registry based: kernels and the GPD tail are declared once, then reused across bundle builders, code generation, MCMC, and S3 methods. For advanced extensions, the practical rule is to stabilize the registry metadata first, then rely on the shared bundle and runner layers rather than writing workflow-specific code.

Advanced extension path

Registry stage: `get_kernel_registry()` and `get_tail_registry()` define signatures, support, defaults, and GPD compatibility.

$\Downarrow$

Bundle stage: builders normalize `fixed`/`dist`/`link` plans into a compiled specification.

$\Downarrow$

Runner stage: manual runners compile NIMBLE code, run MCMC, and attach shared fit metadata.

$\Downarrow$

Consumer stage: `summary()`, `params()`, `predict()`, and `plot()` automatically consume the same fit structure.

Figure 9: Detailed advanced extension path showing how registry metadata propagates through bundle compilation, MCMC execution, and S3 fit consumers. This map is intended for advanced customization without workflow-specific forks.

Layer	Main functions	What advanced users can customize
Registry layer	`get_kernel_registry()`, `get_tail_registry()`	Available kernels, support declarations, bulk parameter names, default covariate-link behavior, GPD compatibility, and backend signatures.
Bundle layer	`build_nimble_bundle()`, `build_causal_bundle()`	Parameter plans, monitoring rules, MCMC defaults, arm-specific causal overrides, and the exact compiled specification passed into NIMBLE code generation.
Runner layer	`run_mcmc_bundle_manual()`, `run_mcmc_causal()`	Parallel execution strategy, worker counts, timing collection, and latent-label update cadence.
Fit-consumer layer	`summary()`, `params()`, `predict()`, `plot()`	Shared summaries that automatically expose new kernels or tail components once the lower layers remain structurally consistent.

Table 16: Advanced extension points across registry, bundle, runner, and fit-consumer layers. The table lists where to modify behavior while preserving compatibility with shared compilation and post-processing infrastructure.

Affiliation:

Firstname Lastname
Affiliation
Address, Country
E-mail: name@address
URL: http://link/to/webpage/

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd